Reglas de estilo, protocolo de programación y protocolo de objetos en Angular

Reglas de estilo, protocolo de programación y protocolo de objetos en Angular

- 1. Reglas de estilo
  - 1.1 .html
    - 1.1.1 Clases
    - 1.1.2 Instancia de objetos
    - 1.1.3 Espaciado
    - 1.1.4 Fechas
  - 1.2 .less
    - 1.2.1 Espaciado
  - 1.3 component.ts
    - 1.3.1 Espaciado
- 2. Protocolo de programación
  - 2.1 Configuración básica
  - 2.2 .html
  - 2.3 .less
    - Cuando usar `position: absolute`
    - Cuando usar `display: flex`
    - Como dar márgenes o espacios
    - Cuando usar `float: left` o `float: right`
  - 2.4 component.ts
    - 2.4.1 Efectos
      - Entrada suave
      - Entrada pop
      - Entrada pop con rebote
      - Entrada desde abajo
      - Entrada desde arriba
      - Entrada desde la izquierda
      - Entrada desde la derecha
    - 2.4.2 Backbutton (boton atrás en Android)
    - 2.4.3 Statusbar
    - 2.4.4 Filtrar por Texto
    - 2.4.5 Validacion Campos
      - CodigoPostal
      - Telefonos
      - Fecha
      - Otros
  - 2.6 service.ts
  - 2.7 Aspectos varios
    - Generación de objetos (componentes)

Objeto particular
Objeto particular simple
Objeto particular complejo
Objeto común
3. Objetos y elementos comunes
3.1 Elementos
3.1.1 Enviar
html
less
ts
3.1.2 Checklist
html
less
ts
3.1.3 Cabeceras
Atrás
Título
Validar, guardar ...
Boton de crear (+), boton de menu (3puntos)
3.1.4 Capa deshabilitadora
Oscura
html
less
Clara
html
less
3.1.5 Boton Switch
html
less
ts
3.1.6 Boton validar
html
less
3.1.7 Boton Enviar y atras
html
less
3.1.8 Campo Entrada
Movil
html
less
Tablet
html
less
3.1.9 Botoneras
3.1.9.1 Tipo 1
html
less
component
3.1.10 Chip

- Simple
  - html
  - less
- Doble
  - html
  - less
- 3.1.11 Cajas
  - html
  - less
- 3.1.12 Listado
  - html
  - less

3.2 Objetos
- 3.2.1 Emergentes
- 3.2.2 Alertas `app-alert`
  - Cómo funciona
    - Lanzar
    - Captura de respuesta
  - Tipos de alerta
    - Tipo 3
    - Tipo 4
    - Tipo 5
  - Creación de nuevas alertas
- 3.2.3 FechahoraPicker `app-fechahora-picker`
  - Atributos propios del calendario
  - Atributos propios de la hora
- 3.2.4 Cargando `app-cargando`
- 3.2.5 Firma `app-firma`
- 3.2.6 Galeria `app-galeria`
- 3.2.7 VisorMedia `app-visor-media`
- 3.2.8 Desplegable `app-desplegable`
- 3.2.9 Pizarra `app-pizarra`
- 3.2.10 Color `app-color`
- 3.2.11 Resumen Enviado
- 3.2.12 Whatsapp `app-whatsapp`
  - `app-entrada`
  - `app-mensajeria`
  - `app-mail`

4. Protocolo multimedia (assets)

4.1 Imágenes
- 4.1.1 Redimensionado
  - ¿Como se mide la imagen?
- 4.1.2 Compresión

Actualizaciones

Este documento detalla las pautas a seguir a la hora de desarrollar en Angular. Es muy importante seguir este documento al pie de la letra porque los proyectos que llevamos a cabo suelen realizarse entre varios programadores.

1. Reglas de estilo

Todo el código, sea el archivo que sea, debe indentarse con 3 espacios **¡OJO!** esto no es un tabulador de tres espacios, sino tres espacios simples, por ello debemos configurar nuestro editor para que al pulsar tabulador, introduzca 3 espacios. Esta configuración en Sublime Text es la siguiente: `tab_size: 3` y `translate_tabs_to_spaces: true`

1.1 .html

1.1.1 Clases

Si queremos nombrar una clase con varias palabras, deberemos hacerlo separando las palabras con un guión. Por ejemplo: `capa-deshabilitadora`, `contenido-boton`.

Dentro de un mismo div, como mucho puede haber otro sin sangrar, lo voy a mostrar con un ejemplo:

Esto **si** esta permitido porque dentro del div solo hay un elemento:

```
<div class="icono"><i class="fas fa-comment"></i></div>
```

Esto **no** esta permitido porque dentro del div hay dos elementos más:

```
<div class="icono"><i class="fas fa-comment"><b>Hola</b></i></div>
```

Por lo tanto debería ser así:

```
<div class="icono">
    <i class="fas fa-comment">
        <b>Hola</b>
    </i>
</div>
```

1.1.2 Instancia de objetos

Cuando instanciamos objetos, los atributos que este tenga, no debe obligarnos ha hacer scroll horizontal, por ello debemos poner los atributos uno debajo del otro de la siguiente manera, recuerda que esto solo es necesario si hay una cantidad de atributos suficiente como para que el código no sea visible sin hacer scroll.

```
<app-objeto class="objeto"
    *ngIf="objeto"
    @efecto
    [ngClass]="{'clase': condicion}"
    [ngStyle]="{'estilo': condicion}"
    [variable1]="0"
    (eventoAlgo1)="ejecutarEvento($event)"
    (eventoCerrar)="cerrar()" >
</app-objeto>
```

El orden de los atributos es importante, observar que `class` siempre va el primero, además siempre tiene que ir en la misma linea que el objeto. Despues siguen los atributos como se muestran arriba

> Obviamente no todos los atributos son obligatorios

1.1.3 Espaciado

Es importante poner los espacios justos en los sitios justos. Nunca debe haber más de un espacio seguido y nunca debe haber un espacio en un sitio que no tiene por que haber espacio, aunque no afecte a la compilación.

Voy a mostrar unos ejemplos de malos espacios, espacios que faltan o sobran (marco los espacios con +):

Dos espacios que no pintan nada:

```
<div+class="boton+cerrar"++(click)="eventoCerrar.emit()"> <!-- MAL -->
<div+class="boton+cerrar"+(click)="eventoCerrar.emit()">  <!-- BIEN -->
```

Un espacio olvidado al final:

```
<div+class="boton+cerrar"+(click)="eventoCerrar.emit()"+> <!-- MAL -->
<div+class="boton+cerrar"+(click)="eventoCerrar.emit()">  <!-- BIEN -->
```

Sin espacio delante de `(click)` donde sí debería haver un espacio:

```
<div+class="boton+cerrar"(click)="eventoCerrar.emit()">  <!-- MAL -->
<div+class="boton+cerrar"+(click)="eventoCerrar.emit()"> <!-- BIEN -->
```

Espacios no necesarios dentro de `div`, si queremos añadir un espacio en blanco lo haremos con ` `:

```
<div+class="nemo">+Petición de Tutoría++</div> <!-- MAL -->
<div+class="nemo">Petición de Tutoría</div>    <!-- BIEN -->
```

Espacios que sobran dentro de `(click)`:

```
<div+class="boton+cerrar"+(click)="+eventoCerrar.emit()"> <!-- MAL -->
<div+class="boton+cerrar"+(click)="eventoCerrar.emit()">  <!-- BIEN -->
```

Lo siguiente esta prohibido, entre las expresiones condicionales tiene que haber un único espacio:

```
<span+*ngIf="obtenerAlumnoSeleccionado()!==0">   <!-- MAL -->
<span+*ngIf="obtenerAlumnoSeleccionado()+!==+0"> <!-- BIEN -->
```

1.1.4 Fechas

Por defecto las fechas deben mostrarse con el siguiente formato:

```
DD/MM/AAAA - HH:MM ( 23/11/2020 - 10:32 )
```

A continuación mostramos otros formatos aceptados:

```
DD/MMM/AAAA - HH:MM ( 23/NOV/2020 - 10:32 )
```

```
DD de M**M a las HH:MM ( 23 de Noviembre a las 10:32 )
```

1.2 .less

Nunca debe haber letras mayúsculas.

1.2.1 Espaciado

Después de cada sentencia hay un único espacio, por ejemplo:

```
width: 100%;
```

Debemos dejar un linea vacía antes de cada hijo, por ejemplo:

```
:host{
   display: block;
   width: 100%;
   height: 100%;
LINEA VACÍA ------------------------------
   .cabecera{
      width: 100%;
      height: 50px;
      position: relative;
   }
}
```

Entre elementos del mismo nivel, no se deja ninguna linea vacía.

1.3 component.ts

1.3.1 Espaciado

Al igual que en los demás ficheros, el espaciado es fundamental para la correcta comprensión del código.

A continuación voy a poner ejemplos de como debe estar espaciado el código. Prestar mucha atención que que así es exactamente como y donde habrá que poner los espacios. Estos son los espacios exactos:

Declaración de variables:

```
disponibilidad: any = [];
```

Las declaraciones de funciones:

```
comprobarValidez() {
    // CODIGO
}

mostrarOpciones($event, item, itemIndex) {
    // CODIGO
}
```

Ifs de una línea:

```
return this.fechas.modo !== 0 && this.obtenerAlumnoSeleccionado() !== undefined;
```

Ifs normales:

```
if ( !this.enviado ) {
    // CODIGO
} else {
    // CODIGO
}
```

Suscripciones:

```
this.sub.sus = this.servicio.sus.subscribe(data => {
    // CODIGO
});
```

Intervalos y Timeouts:

```
setTimeout(() => {
    // CODIGO
}, 300);

setInterval(() => {
    // CODIGO
}, 300);
```

2. Protocolo de programación

2.1 Configuración básica

Cuando creamos un nuevo proyecto se creará `ng new NOMBREPROYECTO --skip-tests`

Cada vez que generamos un `component`, `module`, o `service`, tenemos que añadir la opción `--skip-tests` para que no se generen archivos de test

Necesitamos configurar el proyecto para que podamos importar componentes y modulos con rutas absolutas, para ello tenemos que hacer las siguientes modificaciones en el fichero `tsconfig.json`

Reemplazamos la linea que tiene `baseUrl…` por la siguiente:

```
"baseUrl": "src"
```

A continuación de esa linea añadimos lo siguiente:

```
"paths": {
   "@app/*": ["app/*"]
}
```

Hay que sustituir el fichero styles.less por el fichero que se aporta.

2.2 .html

Es obligatorio comentar todos los `div` que tenga un tamaño considerable y también hay que comentar los divs que sean difíciles de entender a simple vista.

Dentro de los objetos, no es necesario crear una capa bastidor, a no ser que se necesite para una situación concreta. Para poder dar estilo al objeto se debe utilizar `:host` en el `.less`.

Cuando se instancia un objeto, solo se debe meter dentro de una capa si necesita estilos adicionales.

2.3 .less

Debido a una peculiaridad en iOS, todos los `overflow: auto` irán de la siguiente manera, es la única declararción que se deja hacer en una linea:

```
overflow: auto; -webkit-overflow-scrolling: touch;
```

Hay que tener mucho cuidado a la hora de dar estilo, cada situación requiere sus reglas y hay que prevenir usar siempre las mismas reglas para todo, a continuación hay unas pautas para aprender cuando y como deberían usarse ciertas reglas.

Cuando usar `position: absolute`

Esta regla afecta a la propia capa en la que se usa. Solamente debe utilizarse `position: absolute` cuando queremos que una capa se encuentre por encima de otra o si sabemos que una capa puede llegar a estar por encima de otra.

Si una capa lleva `position: absolute`, obligatoriamente tiene que haber una capa padre con `position: relative` para indicar a que capa es relativa.

Todas las capas que llevan `position: absolute` obligatoriamente tienen que llevar al menos una coordenada horizontal `left` o `right`, y una coordenada vertical `top` o `bottom`.

Cuando usar `display: flex`

Esta regla afecta a los hijos de la capa donde se usa. Su principal uso, es para alinear los hijos que lleva dentro, para ello esta regla necesita incluir otras reglas como `justify-content`, `align-items` y `flex-direction`, entre otras.

- `justify-content`: alinea los hijos horizontalmente si `flex-direction: row` y verticalmente si `flex-direction: column`.

- `align-items`: alinea los hijos verticalmente si `flex-direction: row` y horizontalmente si `flex-direction: column`.
- `flex-direction`: indica la dirección en la que se alinean los hijos.

Esta regla también es util en diseños reposive, en el 99% de los casos debe ir acompañada de alguna reglas anteriormente descritas, asi que si vas a usar estar regla 'sola' tienes que tener muy claro por que lo haces

Como dar márgenes o espacios

Por lo general las capas hijos tienen que ocupar el 100% del espacio de la capa padres, si queremos que haya espacios, se hace con `padding` o con `margin`.

Cuando usar `float: left` o `float: right`

Cando tienes varios objetos y quieres que esten uno al lado del otro horizontalmente, debes usar `float: left` o `float: right`. También puedes conseguir el mismo efecto añadiendo estas dos reglas en cada elemento: `display: inline-block` y `vertical-align: top`.

2.4 component.ts

Es obligatorio comentar todas las funciones excepto casos muy obvios donde el nombre de la función ya la explica.

Las variables deben declararse de la siguiente manera:

- Las variables globales van siempre en mayúscula
- Las variables locales van siempre en minúsculas. Las variables que se formen por varias palabras deben estar separadas por un guión bajo, por ejemplo: `constructor_fechas`, `height_listado_fechas`, etc...
- Los eventos `@Output ()` se declaran en minúsculas. Los que se formen por varias palabras deben escribirse juntas con la primera letra de cada palabra en mayúsculas, excepto la primera, por ejemplo: `eventoCerrar`, etc...
- Las comillas que utilizamos dentro de las funciones son siempre comillas simples.
- Cuando declaramos una variable y no va ha cambiar su valor se declara con conts y no con let.

El orden de las variables debe ser el siguiente:

```
@Input()
.
.
@Output()
.
.
@ViewChild()
.
.
VARIABLES_GLOBALES (por ejemplo: BASE_APP)
.
.
variables_locales
```

Todas las variables deben declararse con el tipo para el que vayan a usarse, estos son algunos de ellos:

`array: any = [];`

`objeto: any = {};`

`numero: number;`

`texto: string;`

`boleano: boolean;`

No debemos inicializar variables en la función `ngOnInit`, todo lo que debamos inicializar lo haremos en la propia declaración. Es posible que en algún caso específico, si debamos inicializar una variable en `ngOnInit` pero esto pasará en casos muy particulares.

Todos los objetos que tengan suscripciones, deben declarar la variable `sub: any = {}` donde guardaremos la suscripción para luego en la función `ngOnDestroy` realizar una desuscripción.

> Todas, absolutamente todas las suscripciones a variables debe desuscribirse.

Todos los objetos que necesiten emitir un evento de cerrar, declararán el `Output` `eventoCerrar`:

```
@Output() eventoCerrar = new EventEmitter();
```

Si un objeto necesita acceder a las variables globales del proyecto, lo importará de la siguiente manera:

```
import { Globals } from '@app/globals';
```

2.4.1 Efectos

La mayor parte de los efectos que se utilizan, son idénticos en todos los objetos, por ello para mantener la coherencia, a continuación estan los efectos que ya estan definidos. Está prohibido usar el mismo efecto con otro nombre.

Todos estos efectos están recogidos en el archivo `animations.ts`, para importar los efectos, se hace de la siguiente manera:

```
import { PopUp, inOutFade, inOutFromBottom } from '@app/animations';

@Component({
   .
   .
   .
   animations: [ PopUp, inOutFade, inOutFromBottom ]
})
```

Observar que solo se importan los efectos necesarios

Entrada suave

```
trigger('inOutFade', [
    transition(':enter', [
        style({opacity: '0'}),
        animate('400ms', style({opacity: '1'}))
    ]),
    transition(':leave', [
        animate('400ms', style({opacity: '0'}))
    ])
])
```

Entrada pop

```
trigger('PopUp', [
    transition(':enter', [
        style({transform: 'translate(0, 0) scale(0)'}),
        animate('250ms ease', style({transform: 'translate(0, 0) scale(1)'}))
    ]),
    transition(':leave', [
        style({transform: 'translate(0, 0) scale(1)'}),
        animate('250ms ease', style({transform: 'translate(0, 0) scale(0)'}))
    ]),
])
```

Entrada pop con rebote

```
trigger('PopUpRebote', [
    transition(':enter', [
        style({transform: 'translate(0, 0) scale(0)'}),
        animate('0.4s cubic-bezier(0.175, 0.285, 0.32, 1.1)', style({transform:
'translate(0, 0) scale(1)'}))
    ]),
    transition(':leave', [
        style({transform: 'translate(0, 0) scale(1)'}),
        animate('0.4s cubic-bezier(0.6, -0.28, 0.735, 0.045)', style({transform:
'translate(0, 0) scale(0)'}))
    ]),
])
```

Entrada desde abajo

```
trigger('inOutFromBottom', [
    transition(':enter', [
        style({transform: 'translateY(100%)'}),
        animate('400ms ease', style({transform: 'translateY(0%)'}))
    ]),
    transition(':leave', [
        animate('400ms ease', style({transform: 'translateY(100%)'}))
    ])
])
```

Entrada desde arriba

```
trigger('inOutFromTop', [
    transition(':enter', [
        style({transform: 'translateY(-100%)'}),
        animate('400ms ease', style({transform: 'translateY(0%)'}))
    ]),
    transition(':leave', [
        animate('400ms ease', style({transform: 'translateY(-100%)'}))
    ])
])
```

Entrada desde la izquierda

```
trigger('inOutFromLeft', [
    transition(':enter', [
        style({transform: 'translateX(-100%)'}),
        animate('400ms ease', style({transform: 'translateX(0%)'}))
    ]),
    transition(':leave', [
        animate('400ms ease', style({transform: 'translateX(-100%)'}))
    ])
]),
```

Entrada desde la derecha

```
trigger('inOutFromRight', [
    transition(':enter', [
        style({transform: 'translateX(-100%)'}),
        animate('400ms ease', style({transform: 'translateX(0%)'}))
    ]),
    transition(':leave', [
        animate('400ms ease', style({transform: 'translateX(-100%)'}))
    ])
]),
```

Durante el desarollo de los proyectos, siempre surgen efectos propios de ese proyecto, eso es normal, pero se exige que si se crea un efecto nuevo, ese efecto lleve el mismo nombre en todo el proyecto.

Hay libertad de crear todos los efectos que se necesiten, es importante que el nombre del efecto describa la acción del mismo.

Si fuera necesario utilizar un efecto de los ya definidos, pero con alguna pequeña modificación, el nuevo efecto resultante deberá llevar un nombre compuesto por el nombre original mas una descripción del nuevo cambio, por ejemplo, si necesitamos utilizar el efecto `inOutFromRight` pero queremos cambiar el `translate` de `%` a `vw`, el nombre del nuevo efecto resultante sería `inOutFromRightVw`.

2.4.2 Backbutton (boton atrás en Android)

Todos los objetos deben contemplar el `backbutton` (atrás) de los dispositivos Android, por lo tanto todos los objetos que necesiten emitir un evento de cerrar, tendrán que implemementar la captura y manejo del `backbutton`:

```
@HostListener('document:backbutton', ['$event'])
onBackButton() {
    if ( hijo.visible ) {
        /*
         * Aqui hay dos opciones:
         *
         * Opcion 1:
```

```
         * // Yo como padre me encargo de matar al hijo
         * hijo.visible = false
         * this.cdRef.detectChanges();
         * this.cdRef.reattach();
         *
         * Opcion 2:
         * // No hacer nada y dejar al hijo que capture el backbutton, es decir,
         * // realizar esta misma funcion dentro del hijo. Nos interesa esta
opcion
         * // si el hijo a su vez tiene otros hijos (nietos).
         */
    } else {
        this.cdRef.detectChanges();
        this.cdRef.reattach();
        this.eventoCerrar.emit();
    }
}
```

Tener en cuenta que si el objeto instancia hijos, tendrá que esperar que estos hijos realicen su `eventoCerrar` antes de que el padre se cierre a si mismo.

2.4.3 Statusbar

Cuando sea cordova, hay que tener en cuenta que el objeto que se instancie tiene que cambiar el color de la barra de estado un función del color de la cabecera.

```
import { BarraestadoService } from '@app/utils/barraestado/barraestado.service'

 constructor(
      private barra: BarraestadoService
   ) { }
 ngOnInit() {
    this.barra.establecerBarraEstiloPersonal(false, '#F0F0F0');
 }
```

> Si el padre tiene la cabecera ej. roja y llama a un hijo que tiene la cabecera azul, cuando el hijo se cierra, el padre tiene que volver a poner la cabera roja.

2.4.4 Filtrar por Texto

Esto se utiliza para filtrar por texto dentro de un array

```
    leerAlumnos(curso) {
        this.servicioAlumnos.leerAlumnos(curso).subscribe(data => {
            data.map(item => {
```

```
            item.codigocurso = curso;
            item.nombre_filtro = (item.nombre + ' ' + item.apellido1 + ' ' +
item.apellido2).toLowerCase();
            this.alumnos.push(item);

            this.alumnos_filtrado = this.alumnos;
            this.alumnos_filtrado.sort((a,b) =>
a.nombre_filtro.localeCompare(b.nombre_filtro));
        });
    }
   filtrarPorNombre(alumnos) {
      if ( this.nombre !== undefined && this.nombre !== '' ) {
         return alumnos.filter(alumno => {
            if ( alumno.nombre_filtro.indexOf(this.nombre.toLowerCase()) !== -1 )
{
               return alumno
            }
         })
      } else {
         return alumnos;
      }
   }
```

2.4.5 Validacion Campos

Las siguientes funciones indican como validar diferentes tipos de campos.

CodigoPostal

```
function validateCp(c: FormControl) {
   if ( c.value !== undefined ) {
      if ( c.value.length !== 5 ) {
         return {
            validateCp: {
               valid: false
            }
         };
      }
      else {
         return null;
      }
   }
}
```

Telefonos

```
function validateTlfMovil(c: FormControl) {
   if ( c.value !== undefined ) {
```

```
        if ( c.value.toString().length !== 9 ) {
            return {
                validateTlfMovil: {
                    valid: false
                }
            };
        } else {

            // El teléfono debe empezar por 6 o por 7
            if ( c.value.toString().charAt(0) !== '6' && c.value.toString().charAt(0)
!== '7' ) {
                return {
                    validateTlfMovil: {
                        valid: false
                    }
                };
            } else {
                return null;
            }
        }
    }
}
function validateTlfFijo(c: FormControl) {
    if ( c.value !== undefined ) {
        if ( c.value.toString().length !== 9 ) {
            return {
                validateTlfMovil: {
                    valid: false
                }
            };
        } else {

            // El teléfono debe empezar por 9
            if ( c.value.toString().charAt(0) !== '9' ) {
                return {
                    validateTlfMovil: {
                        valid: false
                    }
                };
            } else {
                return null;
            }
        }
    }
}
function validateTlf(c: FormControl) {
    if ( c.value !== undefined ) {
        if ( c.value.toString().length !== 9 ) {
            return {
```

```
            validateTlfMovil: {
               valid: false
            }
         };
      } else {

         // El teléfono debe empezar por 6 o por 7 o por 9
         if ( c.value.toString().charAt(0) !== '9' && c.value.toString().charAt(0)
!== '6' && c.value.toString().charAt(0) !== '7' ) {
            return {
               validateTlfMovil: {
                  valid: false
               }
            };
         } else {
            return null;
         }
      }
   }
}
```

Fecha

```
function validateFecha(c: FormControl) {
   if ( c.value !== undefined ) {
      if ( c.value.length !== 10 ) {
         return {
            validateFecha: {
               valid: false
            }
         };
      } else {
         if ( !c.value.match(/^(0?[1-9]|[12][0-9]|3[01])[\/](0?[1-9]|1[012])[\/]
((1|2)\d{3})/g) ) {
            return {
               validateFecha: {
                  valid: false
               }
            };
         } else {
            return null;
         }
      }
   }
}
```

Otros

```
function validateRequired(c: FormControl) {
   if ( c.value !== undefined && c.value !== '' ) {
      return null;
   } else {
      return {
         validateRequired: {
            valid: false
         }
      };
   }
}
function validateNull(c: FormControl) {
   return null;
}
```

2.6 service.ts

Todas, absolutamente todas las funciones deben llevar un comentario explicando lo que hacen, los comentarios no pueden ser más anchos que la mitad de la pantalla, para ello usaremos varias lineas si fuera necesario, además el comentario debe tener la siguiente estructura:

```
/*
 * nombreDeLaFuncion: comentario
 */
LINEA VACÍA -------------------------------
nombreDeLaFuncion() {}
```

En el fichero funciones_general.ts que se aporta, encontraras funciones generales que se suelen utilizar generalmente en los proyectos, en estas funciones se indican donde deben incluirse y su funcionalidad.

2.7 Aspectos varios

Generación de objetos (componentes)

Antes de generar un objeto (componente) es necesario pararse a pensar el destino que se le dará a dicho objeto.

Los objetos (componentes) se generan con el siguiente comando:

```
ng g c NOMBRE --skip-tests
```

Aunque siempre es preferible que el nombre sea lo mas sencillo posible, si el nombre del componente es compuesto, debe separarse por guiones `-`, por ejemplo: `alerta-tipo1`, `planning-compuesto`... etc

Se nos pueden dar varias situaciones:

Objeto particular

Este tipo de objeto se subdivide a su vez en dos:

Objeto particular simple

Este tipo de objeto no va a tener hijos que cuelguen de el, además este objeto solo se instancia desde un mismo sitio, por lo tanto no tendrá ni `module` ni `service`.

Objeto particular complejo

Este tipo de objeto puede tener hijos y además puede ser instanciado desde distintos objetos padre, para ello será obligatorio que tenga `module`. Este objeto solo tendrá `service` si lo necesita.

Los módulos `module` se generan con el siguiente comando:

```
ng g m NOMBRE --flat
```

Los servicios `service` se generan con el siguiente comando:

```
ng g s NOMBRE --skip-tests
```

Objeto común

Si estamos generando un objeto que nos puede valer en cualquier parte del proyecto, debemos seguir varias pautas:

1. Generarlo en la carpeta de `objetos-comunes`.
2. Generar `module.ts`.
3. Dentro del `module.ts` es muy importante **declarar** y **exportar** el componente.

Es importante tener en cuenta que estos objetos serán globales a todo el proyecto, por lo tanto hay que diseñarlos de tal manera. Por ello, está prohibido que estos objetos tengan elementos particulares de un objeto concreto. Estos objetos serán configurables mediante `Input()` para que cada objeto que los llame, puedan configurarlos como proceda.

3. Objetos y elementos comunes

En este apartado se definiran el tipo de objetos y elemntos protocolarizados.

El fichero app.html debe tener una capa contenedora que contenga todo el bastidor. Se llamará app-bastidor.

3.1 Elementos

3.1.1 Enviar

Tendrá 3 estados, enviar, enviando y enviado. El enviando llamará al objeto app-cargando . A continuación mostramos el `html`, `less`, y `ts` de este elemento. Este elemento necesita declarar las siguientes variables para su correcto funcionamiento: `enviando` y `enviado`

html

```
<div class="contenido-boton"
    *ngIf="enviar"
     @inOutEnviarBoton
     [ngClass]="{'enviando': enviando, 'enviado': enviado}">
    <div class="boton" [ngClass]="{'enviando': enviando, 'enviado': enviado}">
        <div class="texto con-icono-izquierdo"
            *ngIf="!enviando && !enviado"
             (click)="procesoEnviar()">
            <i class="fa fa-paper-plane"></i>
            ENVIAR
        </div>
        <div class="texto con-icono-izquierdo" *ngIf="enviado">
            <i class="enviado fas fa-check"></i>
            ENVIADO
        </div>
    </div>
</div>
<app-cargando
   *ngIf="enviando && !enviado"
   [texto]="'Enviando...'">
</app-cargando>
```

less

```
.contenido-boton{
    position: absolute;
    bottom: 0;
    left: 0;
    width: 100%;
    height: 70px;
    padding: 10px 35%;

    &.enviado{
        transform: scale(1.2);
        bottom: 0px;
        opacity: 1;
        animation: sello .5s;
        animation-timing-function: ease-in-out;
    }
```

```
@keyframes sello {
    0% {
        transform: scale(3);
        bottom: 0px;
        opacity: 0;
    }
    100% {
        transform: scale(1.2);
        bottom: 0px;
        opacity: 1;
    }
}

.boton{
    display: flex;
    justify-content: center;
    align-items: center;
    width: 100%;
    height: 100%;
    color: #fff;
    background-color: #1ab726;
    border-radius: 26px;
    transition: all .3s;

    &.enviando{
        color: #1ab726;
        background-color: transparent;

        .texto{
            &.con-icono-izquierdo{
                img{
                    height: 100%;
                    object-fit: contain;
                    margin-right: 10px;
                }
            }
        }
    }
    &.enviado{
        // border: 1px solid #1ab726;
        background-color: transparent;
        color: #1ab726;

        .texto{
            align-items: flex-end;
            font-size: 1.2rem;
        }
    }
```

```
.texto{
    width: 100%;
    height: 100%;
    position: relative;
    display: flex;
    align-items: center;
    justify-content: center;
    font-weight: bold;
    font-size: 1.3rem;

    &.con-icono-izquierdo{
        padding-right: 10px;

        i{
            font-size: 1.2rem;
            margin-right: 25px;
            transition: all .3s;

            &.enviado{
                font-size: 1.8rem;
                margin-bottom: 2px;
            }
        }
        img{
            height: 80%;
            object-fit: contain;
            margin-right: 10px;
        }
    }
    &.con-icono-derecho{
        padding-left: 10px;

        i{
            font-size: 1.2rem;
            margin-left: 25px;
            transition: all .3s;

            &.enviado{
                font-size: 1.8rem;
                margin-bottom: 2px;
            }
        }
        img{
            height: 80%;
            object-fit: contain;
            margin-left: 10px;
        }
    }
}
```

```
    }
}
```

ts

```
procesoEnviar() {
    this.enviando = true;

    setTimeout(() => {
        this.enviando = false;
        this.enviado = true;
    }, 2000);
}
```

3.1.2 Checklist

Esta es la lógica que deben seguir todas las checklist, indicamos `html`, `less`, y `ts`:

html

```
<div class="listado">
    <div class="todos" (click)="seleccionarTodos()">
        <div class="check">
            <i *ngIf="todosSeleccionados()" class="fas fa-check-square
seleccionado"></i>
            <i *ngIf="!todosSeleccionados()" class="far fa-square"></i>
        </div>
        <div class="nombre">
            Todos
        </div>
    </div>

    <div class="alumno" *ngFor="let alumno of alumnos"
(click)="seleccionarAlumno(alumno)">
        <div class="check">
            <i *ngIf="alumno.seleccionInsta" class="far fa-check-square
seleccionado"></i>
            <i *ngIf="!alumno.seleccionInsta" class="far fa-square"></i>
        </div>
        <div class="nombre">
            {{alumno.nombre}} {{alumno.apellido1}}
            <span *ngIf="alumno.apellido2 !== 0"> {{alumno.apellido2}}</span>
        </div>
    </div>
</div>
```

less

```sass
.listado{
    width: 100%;
    height: 90%;

    .todos{
        width: 100%;
        height: 50px;
        padding-left: 20px;

        .check{
            width: 15%;
            float: left;
            height: 100%;
            display: flex;
            justify-content: center;
            align-items: center;
            font-size: 1.6rem;
            transition: all .3s;
            color: grey;

            &.deshabilitado{
                opacity: .5;
            }

            .seleccionado{
                color: #009600;
            }
        }
        .nombre{
            width: 85%;
            height: 100%;
            display: flex;
            justify-content: flex-start;
            align-items: center;
            font-size: 1.2rem;
            float: left;
            font-weight: bold;

            &.espacio{
                margin-left: 15%;
            }

            &.deshabilitado{
                opacity: .5;
            }
        }
    }
    .alumno{
        width: 100%;
```

```
        height: 50px;
        padding-left: 20px;

        .check{
            width: 15%;
            float: left;
            height: 100%;
            display: flex;
            justify-content: center;
            align-items: center;
            font-size: 1.6rem;
            transition: all .3s;
            color: grey;

            &.deshabilitado{
                opacity: .5;
            }

            .seleccionado{
                color: #009600;
            }
        }
        .nombre{
            width: 85%;
            height: 100%;
            display: flex;
            justify-content: flex-start;
            align-items: center;
            font-size: 1.2rem;
            float: left;

            &.espacio{
                margin-left: 15%;
            }

            &.deshabilitado{
                opacity: .5;
            }
        }
    }
}
```

ts

```
/*
 * Selecciona todos los elementos de la lista
 */
seleccionarTodos() {
    if ( this.todosSeleccionados() ) {
```

```
        this.alumnos.map(alumno => alumno.seleccionInsta = false);
    } else {
        this.alumnos.map(alumno => alumno.seleccionInsta = true);
    }
}
/*
 * Selecciona un único elemento de la lista
 */
seleccionarAlumno(alumno) {
    alumno.seleccionInsta = !alumno.seleccionInsta;
}
/*
 * Devuelve true si todos los elementos de la lista estan seleccionados
 */
todosSeleccionados() {
    return this.alumnos.filter(alumno => alumno.seleccionInsta).length ===
this.alumnos.length;
}
```

3.1.3 Cabeceras

Por lo general las cabeceras de los objetos tendrán un alto de `50px`;

Las cabeceras se componen:

- boton atrás o Home
- Título (no obligatorio)
- validar, guardar, siguiente o boton de crear (+), boton de menu (3puntos)

Atrás

Ira siempre en el lado izquierdo del objeto en la cabecera.

```
position: absolute;
top: 0;
height: 100%;
display: flex;
justify-content: center;
align-items: center;
font-size: 1.2rem;
color: #0088ff;
font-weight: bold;
padding: 0 20px;
```

Título

```
position: relative;
font-weight: 700;
display: flex;
font-size: 1rem;
text-transform: uppercase;
justify-content: center;
align-items: center;
padding-left: 0;
letter-spacing: 1px;
top: 3%;
```

Validar, guardar ...

Ira siempre en el lado derecho del objeto en la cabecera.

El estilo sera igual que el atrás .

Boton de crear (+), boton de menu (3puntos)

incluirá el estilo del validar y el siguiente:

```
font-size: 1.4rem;
-webkit-text-stroke: 2px white;
```

3.1.4 Capa deshabilitadora

Hay dos tipos de capas deshabilitadoras, una `oscura` y una `clara`. Las propiedades de cada una son distintas:

Oscura

Se utiliza esta capa cuando se quiere deshabilitar lo que hay por detrás de un objeto y ademas darle importancia de primer plano al objeto. Esta capa además va acompañada del efecto `inOutFade`.

html

```
<div class="capa-deshabilitadora-oscura" *ngIf="..." @inOutFade></div>
```

less

```
.capa-deshabilitadora-oscura{
    position: fixed;
    top: 0;
    left: 0;
    width: 100vw;
    height: 100vh;
    background-color: rgba(0,0,0,0.7);
    z-index: 1; // Este z-index deberá ir cambiando según convenga
}
```

Clara

Se utiliza esta capa cuando solo se quiere deshabilitar lo que hay por detrás

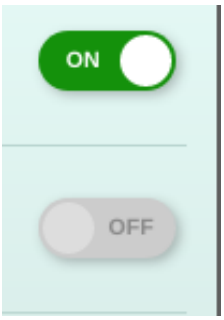
html

```
<div class="capa-deshabilitadora-clara" *ngIf="..."></div>
```

less

```
.capa-deshabilitadora-clara{
    position: fixed;
    top: 0;
    left: 0;
    width: 100vw;
    height: 100vh;
    z-index: 1; // Este z-index deberá ir cambiando según convenga
}
```

3.1.5 Boton Switch



html

```
<div class="enchufe">
    <div class="switch" (click)="item.estado === 0 ? apagar(item.id) :
encender(item.id)">
        <input [(ngModel)]="!item.estado" type="checkbox" id="toggle-button-
checkbox" class="no-pointer-events" >
        <label class="toggle-button-switch" ></label>
        <div class="toggle-button-text">
            <div *ngIf="item.estado===0" class="toggle-button-text-on">ON</div>
            <div *ngIf="item.estado===1" class="toggle-button-text-off">OFF</div>
        </div>
    </div>
</div>
```

less

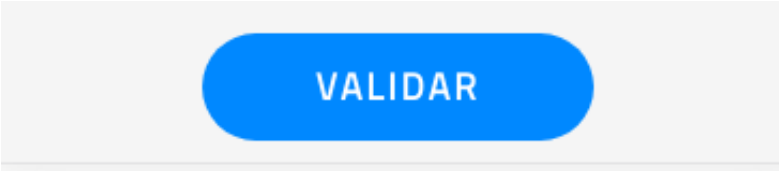
```
.enchufe{
width: 30%;
height: 100%;
position: relative;
display: flex;
justify-content: center;
align-items: center;

    .switch{
    @swidth: 16vw;
    @sheight: 7vw;
    display: block;
    position: relative;
    width: @swidth;
    height: @sheight;
    border: 0;
    input[type="checkbox"] {
        display: none;
    }
    .toggle-button-switch {
        position: absolute;
        top: 2px;
        left: 2px;
        width: calc(@sheight - 4px);
        height: calc(@sheight - 4px);
        border-radius: 100%;
        background-color: #ddd;
        cursor: pointer;
        transition: left 0.3s;
    }
    .toggle-button-text {
```

```
        overflow: hidden;
        background-color: #ccc;
        border-radius: 1000px;
        box-shadow: 2px 2px 5px 0 rgba(50, 50, 50, 0.3);
        transition: background-color 0.3s;
        color: #8a8a8a;
    }
    .toggle-button-text-on,
    .toggle-button-text-off {
        float: left;
        width: 65%;
        height: 100%;
        line-height: @sheight;
        font-weight: bold;
        text-align: center;
        font-size: 1rem;
    }
    .toggle-button-text-off {
        float: right;
    }
    input[type="checkbox"]:checked ~ .toggle-button-switch {
        left: calc(@swidth - (@sheight - 4px) - 2px);
        background-color: #fff;
    }
    input[type="checkbox"]:checked ~ .toggle-button-text {
        background-color: #14910b;
        color: white !important;
    }
}
```

ts

```
apagar(zona) {
    this.estancia.iot[zona-1].estado = 1;
}

encender(zona) {
    this.estancia.iot[zona-1].estado = 0;
}
```

3.1.6 Boton validar



html

```html
<div class="contenido-boton">
    <div class="boton">
        <div class="texto" (click)="validar()">VALIDAR</div>
    </div>
</div>
```

less

```less
.contenido-boton{
position: fixed;
bottom: 0;
left: 0;
width: 100%;
height: 70px;
padding: 10px 25%;
display: flex;
justify-content: center;
align-items: center;
background-color: #f0f0f0;
border-top: 1px solid #dedede;
z-index: 1;
    .boton{
        color: #777777;
        display: flex;
        align-items: center;
        justify-content: center;
        font-size: 1.6rem;
        background-color: white;
        border: 1px solid #dadada;
        font-weight: BOLD;
        border-radius: 1000px;
        padding: 12px 10px;
        transition: all 0.4s;
        font-family: arial;
        width: 100%;
        height: 100%;
    }

}
```

3.1.7 Boton Enviar y atras

html

```
<div class="atras" (click)="validado = false">
    <i class="fas fa-chevron-left"></i>
</div>
<div class="contenido-boton">
    <div class="boton validado">
        <div class="texto" (click)="fucnion()">
            <i class="fa fa-paper-plane"></i>
            ENVIAR
        </div>
    </div>
</div>
```

less

```
.atras{
    position: absolute;
    bottom: 0;
    left: 0;
    width: 70px;
    height: 70px;
    display: flex;
    justify-content: center;
    align-items: center;
    font-size: 2rem;
    padding-left: 5%;
    color: #0088ff;;
    z-index: 2;
}
.contenido-boton{
    position: fixed;
    bottom: 0;
    left: 0;
    width: 100%;
    height: 70px;
    padding: 10px 25%;
    display: flex;
    justify-content: center;
    align-items: center;
    background-color: #f0f0f0;
    border-top: 1px solid #dedede;
```

```
    z-index: 1;

    .boton{
        color: #777777;
        display: flex;
        align-items: center;
        justify-content: center;
        font-size: 1.6rem;
        background-color: white;
        border: 1px solid #dadada;
        font-weight: bold;
        border-radius: 1000px;
        padding: 12px 10px;
        transition: all 0.4s;
        font-family: arial;
        width: 100%;
        height: 100%;

        &.validado{
            background-color: #1ab726;
            color: white;
            i{
                color: white;
            }
        }
    }
}
```

3.1.8 Campo Entrada

Movil

html

```
 <div class="item" #itemComentario (click)="mostrarComentario()">
        <span class="icono comentario">
           <i class="fas fa-pencil-alt"></i>
        </span>
        <div class="titulo">
           <span>COMENTARIO</span>
        </div>
        <div class="datos">
           <span class="dato sin" *ngIf="texto_tutoria === ''">Escribir
comentario</span>
           <span class="dato comentario" *ngIf="texto_tutoria !== ''">
{{texto_tutoria}}</span>
        </div>
      </div>
```

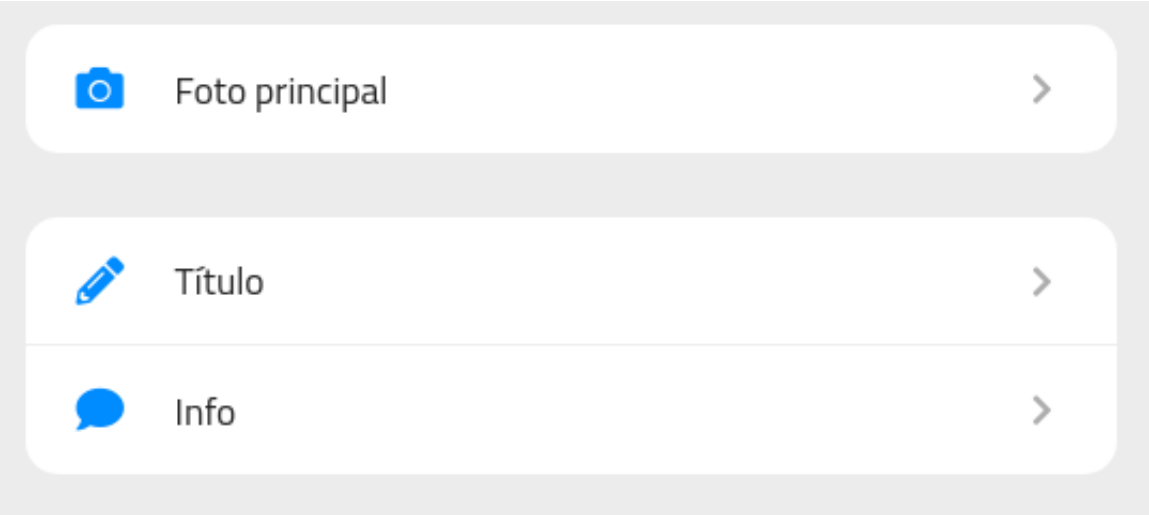
less

```
.item{
    width: 100%;
    min-height: 60px;
    position: relative;
    margin-bottom: 10px;
    float: left;
    background-color: white;
    border-radius: 10px;
    &.error{
        border: 1px solid rgba(255, 0, 0, 0.55);
    }
    &.validado{
        background-color: transparent;

        .icono{
            color: #b1b1b1
        }
    }
    .icono{
        width: 70px;
        height: 60px;
        display: flex;
        justify-content: center;
        align-items: center;
        color: #007eff;
        font-size: 2.3rem;
        float: left;
        padding-bottom: 6px;

        &.comentario{
            -webkit-text-stroke: 1px #f8f8f8;
```

```
        }
    }
    .titulo{
        width: calc(100% - 70px);
        height: 30px;
        float: left;
        font-size: 1.3rem;
        font-family: 'lato orig';
        color: #777;
        display: flex;
        justify-content: flex-start;
        align-items: flex-end;
    }
    .datos{
        width: calc(100% - 70px);
        min-height: 30px;
        position: relative;
        float: left;
        padding-top: 2px;
        padding-bottom: 5px;
        padding-right: 15px;

        &::-webkit-scrollbar {
            display: none;
        }
        &.texto{
              padding-right: 8%;
        }

        .dato{
            color: #505050;
            font-size: 1.3rem;
            word-break: break-all;

            &.sin{
                font-style: italic;
                color: grey;
                font-size: 1.3rem;
            }
            &.texto{
                -webkit-appearance: textfield;
            }
            &.comentario{
                overflow: hidden;
                text-overflow: ellipsis;
                width: 100%;
                display: block;
                white-space: nowrap;
            }
```

```
        }
    }
}
```

Tablet



> En la tablet estos elementos pueden ir solos o en grupo, si van en grupo tendrás dos div elemento.

html

```
<div class="grupo">
      <div class="elemento" (click)="mostrarTituloInfo()">
         <div class="icono">
            <i class="fas fa-pencil-alt"></i>
         </div>
         <div class="texto" *ngIf="tituloinfo === undefined || tituloinfo ===
''">Título</div>
         <div class="texto comentario" *ngIf="tituloinfo !== undefined &&
tituloinfo !== ''">{{tituloinfo}}</div>
         <div class="indicativo"><i class="fas fa-chevron-right"></i></div>
      </div>
      <div class="elemento" (click)="mostrarInfo()">
         <div class="icono">
            <i class="fas fa-comment"></i>
         </div>
         <div class="texto" *ngIf="info === undefined || info === ''">Info</div>
         <div class="texto comentario" *ngIf="info !== undefined && info !== ''">
{{info}}</div>
         <div class="indicativo"><i class="fas fa-chevron-right"></i></div>
      </div>
   </div>
```

less

```sass
.grupo{
        margin-bottom: 30px;
        background-color: white;
        border-radius: 15px;
        width: 100%;
        float: left;
        position: relative;

        &.half{
           width: calc(50% - 20px);

           &.left{
              margin-right: 40px;
           }
        }

        &.validado{
           background-color: transparent;
        }

        &.alineado{
           padding-left: 70px;
        }

        .elemento{
           height: 60px;
           border-bottom: 1px solid #efefef;
           position: relative;

           &:last-child{
              border-bottom: none;
           }

           .icono{
              width: 70px;
              height: 100%;
              float: left;
              padding: 2%;
              display: flex;
              justify-content: center;
              align-items: center;
              color: #008bff;
              font-size: 1.4rem;

              &.verde{
                 color: #1fb51f;
              }
              &.check-fechas{
                 width: 50px;
```

```
            padding-left: 0;
        }

        img{
            width: 100%;
            height: 100%;
            object-fit: contain;
        }
    }
    .texto{
        height: 100%;
        // max-width: calc(100% - 140px);
        float: left;
        display: flex;
        align-items: center;
        font-size: 1.1rem;

        &.sin{
            font-style: italic;
            color: grey;
        }

        &.comentario{
            display: block;
            line-height: 60px;
            white-space: nowrap;
            overflow: hidden;
            text-overflow: ellipsis;
            width: calc(100% - 140px);
        }
        &.imagen{
            height: 100%;
            width: calc(100% - 140px);
            padding: 7px 0;
            display: flex;

            img{
                height: 100%;
                object-fit: contain;
                border-radius: 3px;
            }
        }
    }
    .nombre{
        height: 100%;
        float: left;
        display: flex;
        align-items: center;
        color: #005ca9;
```

```
            text-transform: uppercase;
            font-weight: bold
        }
        .indicativo{
            width: 70px;
            height: 100%;
            float: right;
            display: flex;
            justify-content: center;
            align-items: center;
            color: #adadad;

            &.azul{
                color: #0088ff;
            }
        }
        .select{
            width: calc(100% - 140px);
            height: 100%;
            float: left;

            .opcion{
                width: 50%;
                height: 100%;
                float: left;

                &.otras{
                    border-left: 1px solid #efefef;
                }

                &.validado{
                    .texto{
                        width: auto;
                    }
                    .indicativo{
                        float: left;
                    }
                }

                .check{
                    width: 30%;
                    height: 100%;
                    float: left;
                    display: flex;
                    justify-content: center;
                    align-items: center;
                    font-size: 1.2rem;
                    color: #0088ff;
```

```
                &.seleccionado{
                    font-size: 1.4rem;
                }
            }
            .texto{
                width: 70%;
                height: 100%;
                float: left;
            }
        }
    }
}

.caja{
    border: 1px solid #d5d9dd;
    border-radius: 6px;
    background: whitesmoke;
    box-shadow: 1px 1px 10px #dedede;
    margin: 30px 0;

    .titulo{
        width: 100%;
        height: 35px;
        padding-left: 10px;
        background: #e7e7e7;
        font-weight: bold;
        color: #777;
        border-top-right-radius: 6px;
        border-top-left-radius: 6px;
        display: flex;
        justify-content: left;
        align-items: center;
        position: relative;

        .icono{
            margin-right: 10px;

            &.verde{
                color: #1fb51f;
            }
        }
    }

    .texto{
        width: 100%;
        height: 50px;
        display: flex;
        align-items: center;
        font-size: 1.1rem;
```

```
                padding-left: 5%;

                &.sin{
                    font-style: italic;
                    color: grey;
                }

                &.comentario{
                    display: block;
                    line-height: 50px;
                    white-space: nowrap;
                    overflow: hidden;
                    text-overflow: ellipsis;
                    width: 100%;
                    padding-right: 5%;
                }
            }
        }
```

3.1.9 Botoneras

3.1.9.1 Tipo 1

html

```
<div class="botonera-cursos">
      <div class="boton-curso"
         *ngFor="let curso of cursos"
         (click)="seleccionarCurso(curso)"
         [ngClass]="{'activo': curso.seleccionado}">
         {{curso.tipo}} <span class="total">{{('0' +
curso.totalescurso).slice(-2)}}</span>
      </div>
   </div>
```

less

```
.botonera-cursos{
    display: flex;
    border: 1px solid #49a1eb;
    border-radius: 7px;
    overflow: hidden;

    .boton-curso{
```

```
        padding: 0 13px;
        border-right: 1px solid #49a1eb;
        color: #49a1eb;
        font-weight: bold;
        font-size: .9rem;

        &.activo{
            background-color: #49a1eb;
            color: white;
            font-weight: bold;
        }

        &:last-child{
            border-right: none;
        }
    }
}
```

component

```
 seleccionarCurso(curso) {
     this.cursos.map( curso => curso.seleccionado = false );
     curso.seleccionado = true;
}
```

3.1.10 Chip

Simple

NL 00

html

```
<div class="chip"
     (click)="eventoSeleccion.emit({'opcion': opcion.id, 'tipo': 0});
clickedOutside()"
     [ngClass]="{'activo': opcion.nl !== 0 && opcion.nl !== undefined}">
    <span class="nemo">NL</span>
    <span class="dato">
        <span *ngIf="opcion.nl !== 0 && opcion.nl !== undefined">{{ ("0" +
opcion.nl).slice(-2) }}</span>
        <span *ngIf="opcion.nl === 0 || opcion.nl === undefined">00</span>
        <div class="adorno"></div>
    </span>
</div>
```

less

```less
.chip{
    background-color: #f5f5f5;
    color: #6f6f6f;
    border-radius: 1000px;
    overflow: hidden;

    .nemo{
        padding: 0 10px;
        display: inline-block;
    }
    .dato{
        padding: 0 8px;
        border-radius: 100px;
        display: inline-block;
        position: relative;

        .adorno{
            position: absolute;
            top: -1px;
            right: -1px;
            border: 2px solid white;
            border-radius: 100px;
            width: calc(100% + 2px);
            height: calc(100% + 2px);
        }
    }

    &.activo{
        background-color: rgb(242,250,255);
        color: rgb(42, 143, 202);
        border: 1px solid rgb(42, 143, 202);

        .dato{
            .adorno{
                border: 1px solid rgb(191, 221, 243) !important;
            }
        }
    }
}
```

Doble

NL	00
NC	00

html

```
<div class="estados" (click)="mostrarDesplegable(1)">
    <div class="chip">
        <span class="nemo">NL</span>
        <span class="dato">
            <span *ngIf="buzon.padres.nl !== 0 && buzon.padres.nl !== undefined">
{{ ("0" + buzon.padres.nl).slice(-2) }}</span>
            <span *ngIf="buzon.padres.nl === 0 || buzon.padres.nl ===
undefined">00</span>
            <div class="adorno"></div>
        </span>
    </div>
    <div class="chip">
        <span class="nemo">NC</span>
        <span class="dato">
            <span *ngIf="buzon.padres.lnc !== 0 && buzon.padres.lnc !==
undefined">{{ ("0" + buzon.padres.lnc).slice(-2) }}</span>
            <span *ngIf="buzon.padres.lnc === 0 || buzon.padres.lnc ===
undefined">00</span>
            <div class="adorno"></div>
        </span>
    </div>
 </div>
```

less

```
.estados{
    width: calc(70% - 40px);
    display: flex;
    flex-direction: column;
    justify-content: center;
    align-items: center;

    .chip{
        background-color: #f5f5f5;
        color: #6f6f6f;
        border-radius: 10px;
        overflow: hidden;
        width: 80px;
        display: flex;
        justify-content: space-between;

        &:first-child{
            border-bottom-left-radius: 0px;
            border-bottom-right-radius: 0px;

            .dato{
                .adorno{
                    border-top-left-radius: 10px;
```

```
                    border-top-right-radius: 10px;
                }
            }
        }
        &:last-child{
            border-top-left-radius: 0px;
            border-top-right-radius: 0px;

            .dato{
                .adorno{
                    border-bottom-left-radius: 10px;
                    border-bottom-right-radius: 10px;
                }
            }
        }

        .nemo{
            padding: 0 10px;
            display: inline-block;
        }
        .dato{
            padding: 0 8px;
            border-radius: 100px;
            display: inline-block;
            position: relative;

            .adorno{
                position: absolute;
                top: -2px;
                right: -2px;
                border: 2px solid white;
                width: calc(100% + 4px);
                height: calc(100% + 4px);
            }
        }

        &.activo{
            background-color: rgb(242,250,255);
            color: rgb(42, 143, 202);
            border: 1px solid rgb(42, 143, 202);

            .dato{
                .adorno{
                    border: 1px solid rgb(191, 221, 243) !important;
                }
            }
        }
    }
}
```

3.1.11 Cajas

html

```html
<div class="contenedor">
    <div class="caja"></div>
</div>
```

less

```less
.contenedor{
    background-color: #f3f3f3;

    .caja{
        width: ?;   // Según convenga
        height: fit-content;
        position: relative;
        background-color: white;
        padding: 20px;
        margin: 0 auto;
        border-radius: 8px;
        box-shadow: 0 0 7px #e1e1e1;
    }
}
```

3.1.12 Listado

html

```html
<div class="listado">
    <div class="elemento" *ngFor="let elemento of elementos">

    </div>
</div>
```

less

```less
.listado{
    width: ?;   // Según convenga
    height: ?;  // Según convenga
    background-color: #f3f3f3;
    overflow: auto; -webkit-overflow-scrolling: touch;

    .elemento{
        width: ?;   // Según convenga
        height: ?;  // Según convenga
        position: relative;
        background-color: white;
```

```
        padding: 20px;
        margin: 0 auto;
        border-radius: 8px;
        box-shadow: 0 0 7px #e1e1e1;
    }
}
```

3.2 Objetos

Los objetos comunes estan protocolarizados y no se pueden modificar sin previo aviso.

3.2.1 Emergentes

Los objetos emergentes llevan una capa deshabilitadora oscura y un cerrar arriba.

El boton de cerrar:

```
.boton-cerrar{
    width: 50px;    // Para tablet
    height: 50px;   // Para tablet
    width: 44px;    // Para móvil
    height: 44px;   // Para móvil
    position: absolute;
    top: -25px;     // Libre para cambiar
    right: -25px;   // Libre para cambiar
    display: flex;
    justify-content: center;
    align-items: center;
    background: #fff;
    border-radius: 10000px;
    font-size: 1.4rem;
    border: 1px solid #ccc;
    z-index: 3;
    -webkit-text-stroke: 1px #fff;
}
```

3.2.2 Alertas `app-alert`

Todos los proyectos tienen un sistema centralizado de alertas, a continuación mostramos como se utiliza dicho sistema, además de las distintas alertas que tenemos creadas.

Para que el sistema de funcione, simplemente tenemos que importar en `app.module.ts` el módulo del objeto de alertas que siempre debe encontrarse en `utils/alert`, después declaramos en `app.component.html` al final del fichero la siguiente linea:

```
<app-alert></app-alert>
```

Con esto el sistema de alertas ya debería funcionar si problemas y además funcionará en todo el proyecto.

Cómo funciona

Para lanzar una alerta, debemos importar el servicio de alertas en el componenten desde el que queramos lanzar una alerta

```
import { AlertService } from '@app/utils/alert/alert.service';
```

Siempre se declarará en el `constructor` de la siguiente manera:

```
private servicioAlerta: AlertService
```

Lanzar

Para lanzar alertas, el servicio de alertas tiene un método llamado `mostrarAlerta` al que debemos llamar, por ejemplo:

```
this.servicioAlerta.mostrarAlerta(
    3,  // TIPO DE ALERTA
    {
        texto: 'Es necesario que expliques el motivo',
        botontexto: 'Entiendo'
    }
);
```

En el ejemplo anterior vemos que se esta lanzando la alerta de tipo `3`, (véase Tipos de alerta) indicando el texto del mensaje y el texto del boton de aceptar.

Captura de respuesta

No siempre nos interesa capturar lo que el usuario ha seleccionado dentro de la alerta, pero para cuando si nos haga falta capturar la respuesta, el servicio de alertas cuenta con un observador al que nos podemos suscribir para monitorizar la interacción del usuario, pera ello, en la función `ngOnInit` de nuestro objeto debemos suscribirnos a este observador de la

siguiente manera:

```
this.sub.alerta = this.servicioAlerta.eventoAlerta.subscribe(data => {

});
```

En este caso data será la respuesta del objeto de alerta, todos los tipos de alerta tienen por norma general los siguientes códigos establecidos:

- `data = 0`: significa que se ha cerrado la alerta
- `data = 2`: significa que se ha pulsado el boton de aceptar

Los objetos de alerta pueden tener botones adicionales que lancen codigos distintos, pero los anteriormente descritos siempre tienen que significar lo mismo en cualquier tipo de alerta.

Suele pasar que dentro de un mismo objeto necesitemos lanzar mas de un alerta o mas de un tipo de alerta y que para cada una la acción sea distinta, para poder diferenciar las alertas, debemos declararnos una variable `alert: number`, de manera que cuando lancemos una alerta, ponemos alert a un número específico para que nosotros podamos más tarde diferenciarla. Por ejemplo:

```
ngOnInit() {
    this.sub.alerta = this.servicioAlerta.eventoAlerta.subscribe(data => {
        if ( this.alerta === 1 && data === 2 ) {
            /*
             * Aquí ejecutamos el código perteneciente a la
             * alerta que hemos nombrado con '1'
             */
        } else if ( this.alerta === 2 && data === 2 ) {
            /*
             * Aquí ejecutamos el código perteneciente a la
             * alerta que hemos nombrado con '2'
             */
        }

        this.alerta = 0;
    });
}

lanzarAlerta1() {
    this.alerta = 1;
    this.servicioAlerta.mostrarAlerta(
        3,
        {
            texto: 'Debes indicar el motivo de la anulación',
            botontexto: 'Entiendo'
        }
    );
}
```

```
lanzarAlerta2() {
    this.alerta = 2;
    this.servicioAlerta.mostrarAlerta(
        3,
        {
            texto: 'Debes indicar el motivo',
            botontexto: 'Entiendo'
        }
    );
}
```

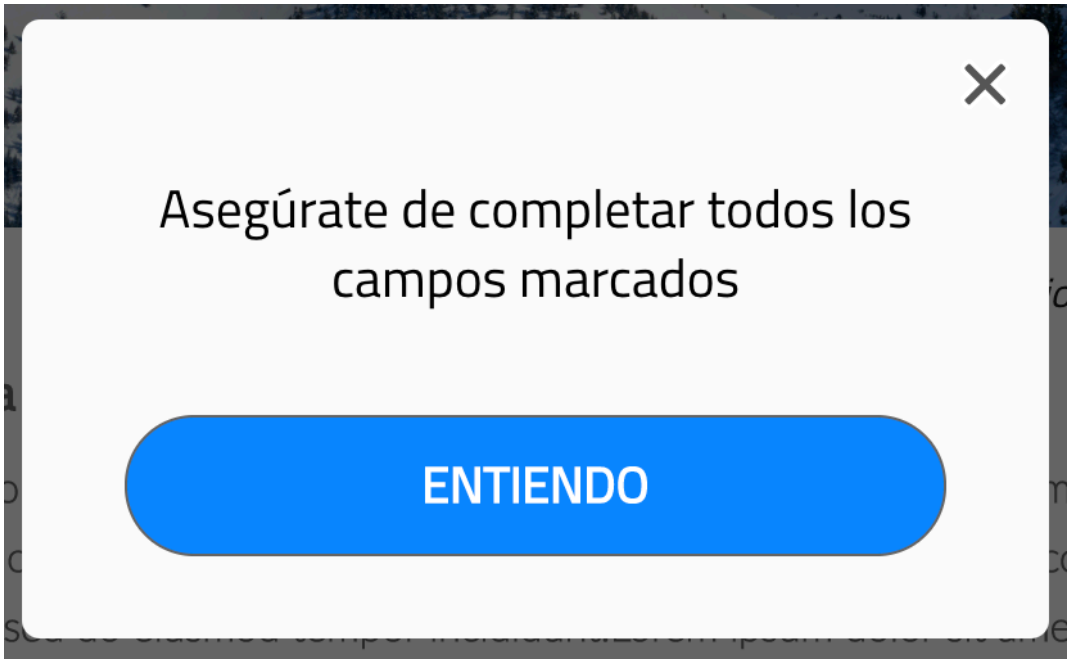
Tipos de alerta

Tipo 3

Este tipo de alerta recibe los atributos:

`texto`

`botontexto`



Tipo 4

Este tipo de alerta recibe los atributos:

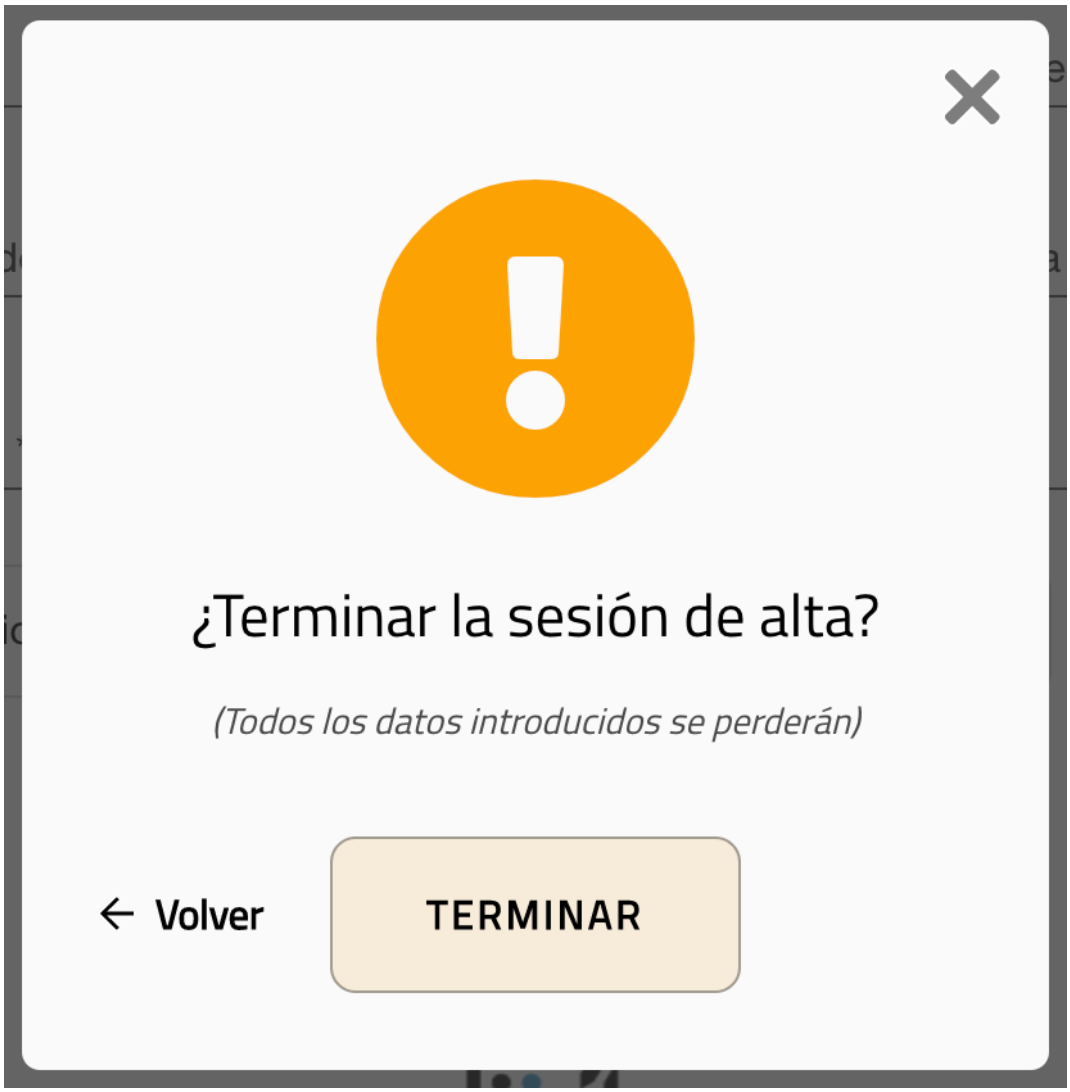
`texto`

`texto1`

`botontextoaceptar`

`botontextocancelar`

En este tipo de alerta es importante procurar que `texto1` no sobresalga al `texto`



Tipo 5

Este tipo de alerta recibe los atributos:

`texto1`

`texto2`

`texto3`

`botontextoaceptar`

`botontextocancelar`

En este tipo de alerta es importante procurar que `texto1` no sobresalga al `texto`

Creación de nuevas alertas

Si se necesitara algún tipo de alerta nueva, se creará siguiendo el orden numérico, además tiene que cumplir ciertas condiciones.

Lo más importante es que todas las alertas deben ser `resposive`, por lo tanto deberán crearse con `px` y utilizar `media-queries`.

Lo segundo es que debe respetarse los códigos de respuesta que anteriormente hemos descrito.

3.2.3 FechahoraPicker `app-fechahora-picker`

Es necesario importar lo siguiente en la cabecera del `index.hml`:

```
<link href="https://fonts.googleapis.com/icon?family=Material+Icons"
rel="stylesheet">
```

Además necesitamos el siguiente plugin para angular, importar en `package.json`:

```
"igniteui-angular": "^8.2.16"
```

Este objeto recibe los siguientes atributos:

`modo` Recibe un numero del 0-2 dependiendo del objeto que deseemos mostrar.

- 0 Calendario y Hora.
- 1 Calendario.
- 2 Hora

Atributos propios del calendario

`@Input() ver_dia: Date`: Recibe el dia que se vera marcado.

`@Input() multi: boolean = false`: Indica si se pueden marcar multiples fechas o no. IMPORTANTE: solo tiene valor si `rango = false`.

`@Input() rango: boolean = false`: Habilitado para poder marcar un rango de fechas.

`@Input() dias_marcados: Date[]`: Días marcados.

`@Input() dias_disponibles: Date[]`: Días disponibles.

`@Input() color_nuevo: string`: Color con el que se marcará los días nuevos. Opciones disponibles `azul`, `naranja`

`@Input() deselect: boolean = false`: Para deseleccionar los dias que estan seleccionados. Tipo variable Boolean.

`@Input() weekend: boolean = false` Esta variable habilita la posibilidad de seleccionar dias de fin de semana.

Atributos propios de la hora

`@Input() hora: string`: Hora que aparera marcada.

`@Input() intervalo: number = 1`: Intervalo en minutos.

`@Input() horas = any []`: Si no enviamos intervalo y queremos que le lleguen las horas en un array.

`@Input() hora_inicio: number = 0`: (Solo cuando se le envia el intervalo)

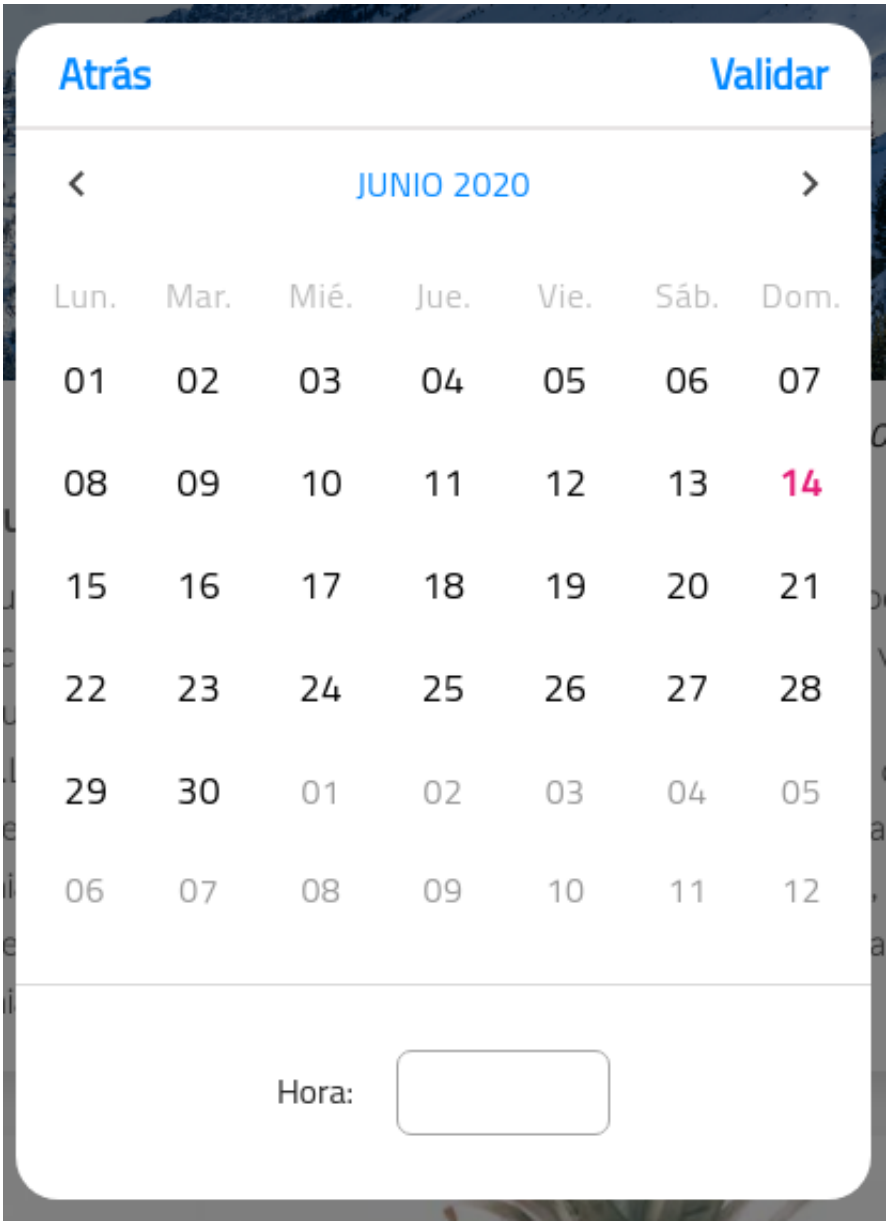
`@Input() hora_fin: number = 24` (Solo cuando se le envia el intervalo)

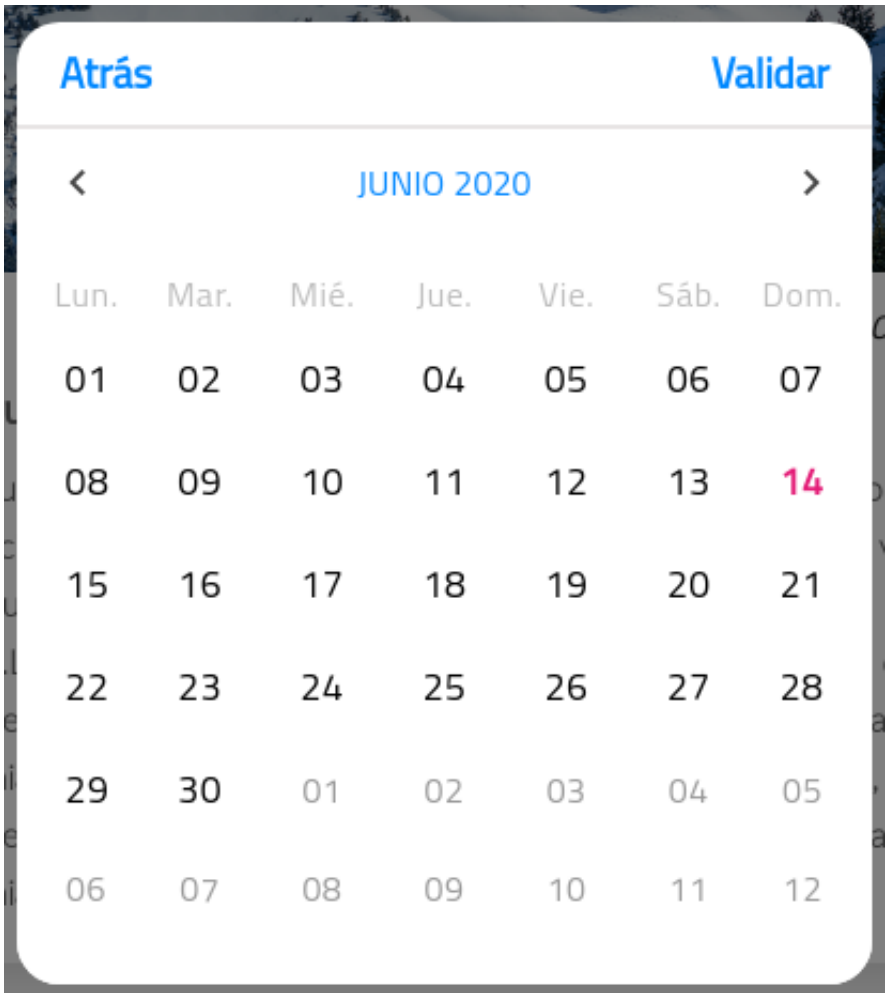
`@Input() hora_min: number` Hora minima. (Solo cuando se le envia el intervalo)

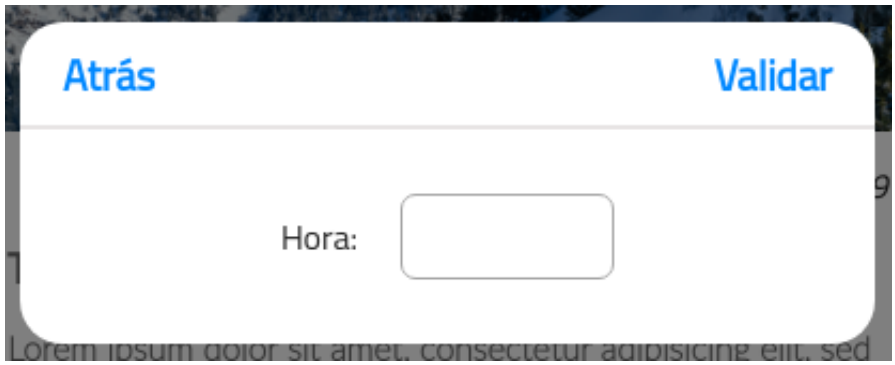
`@Input() minuto_min: number` Minuto Minimo. (Solo cuando se le envia el intervalo)

`@Output() eventoCerrar`

`@Output() eventoValidado` Devuelve un objeto. { dia[], hora}

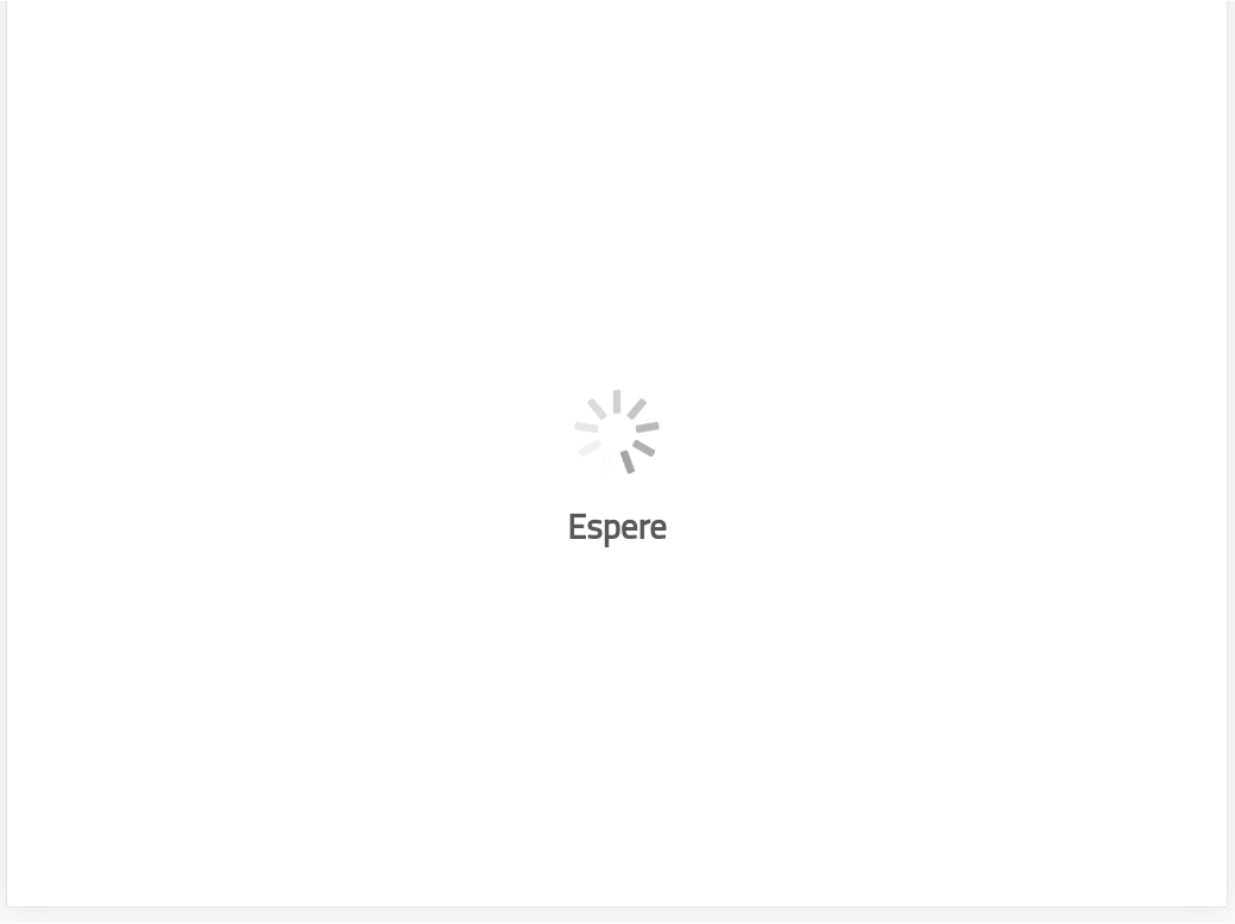






3.2.4 Cargando `app-cargando`

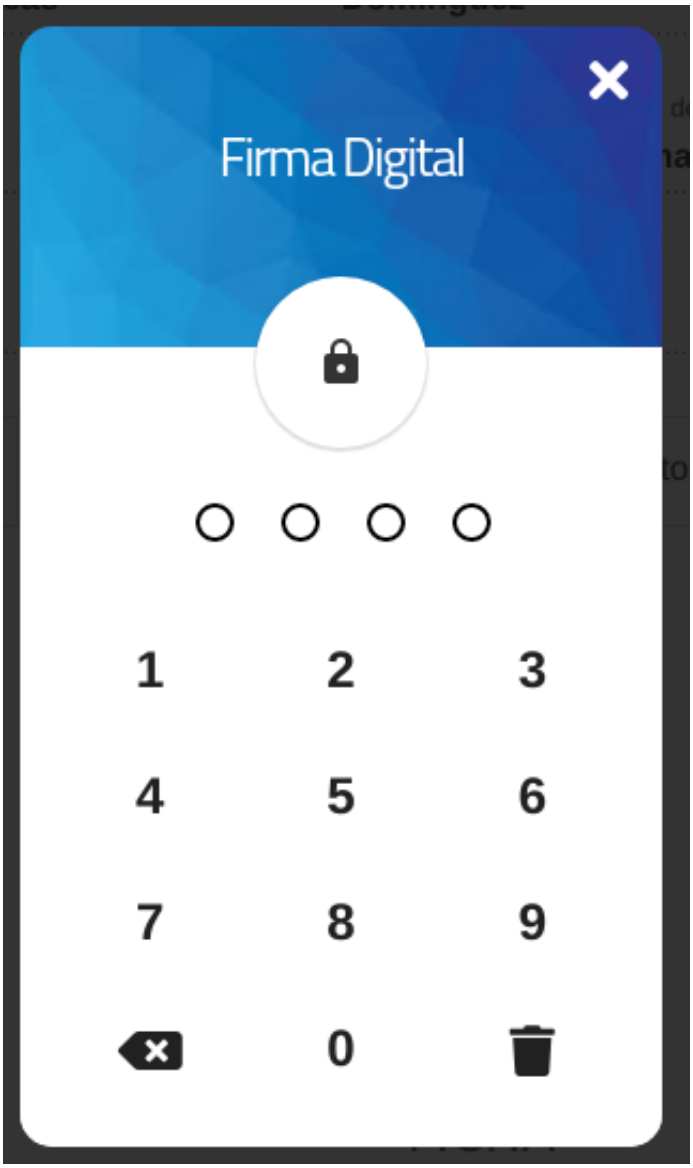
recibe los atributo `@Input() texto: string`



3.2.5 Firma `app-firma`

`Output eventoCorrecto`

`Output eventoCerrar`



3.2.6 Galeria `app-galeria`

Recibe los atributos

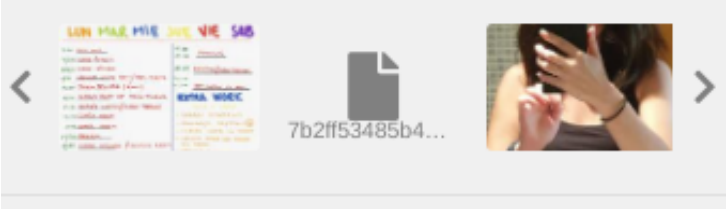
`@Input() adjuntos: any []`

`@Input() eliminable: boolean = false`

`@Input() imprimible: boolean = false`

`@Input() descargable`: boolean = false`

`@Input() readonly`: boolean = false`: No se puede eliminar



3.2.7 VisorMedia `app-visor-media`

Este objeto se utilizar para ver imagenes, videos a pantalla completa y pader hacer zoom en la imgen.

`@Input() tipo` Por defecto es 0 Imagen. Si es video se le enviará un 1.

`@Input() archivo: string`

`@Input() imgComprimido`

`@Output() eventoCerrar`

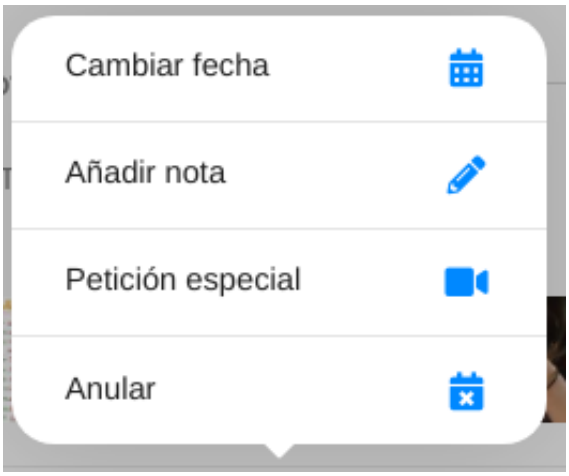
3.2.8 Desplegable `app-desplegable`

`@Input() opciones: any` Tipo variable [id, nemo, icono]

`@Input() indicador: number` 0 izda 1 dcha 2 arriba 3 abajo

`@Output() seleccion` Devuelve el id del array de opciones

`@Output() eventoCerrar`



3.2.9 Pizarra `app-pizarra`

`@Input() formato: Formato` Puede ser `Extenso` `(0)` o `Corto` `(1)`

`@Input() limite: number = 10000` Limite de caracteres

`@Input() texto: string` Texto que vanga ya definido o ya hayan escrito

`@Input() titulo: string`

`@Input() titulo_secundario: sring`

`@Input() cesta_adjuntos: any = []`

`@Input() adjuntos: boolean = true`

`@Input() readonly:boolean = false`

`@Input() nullable:booleam = true`

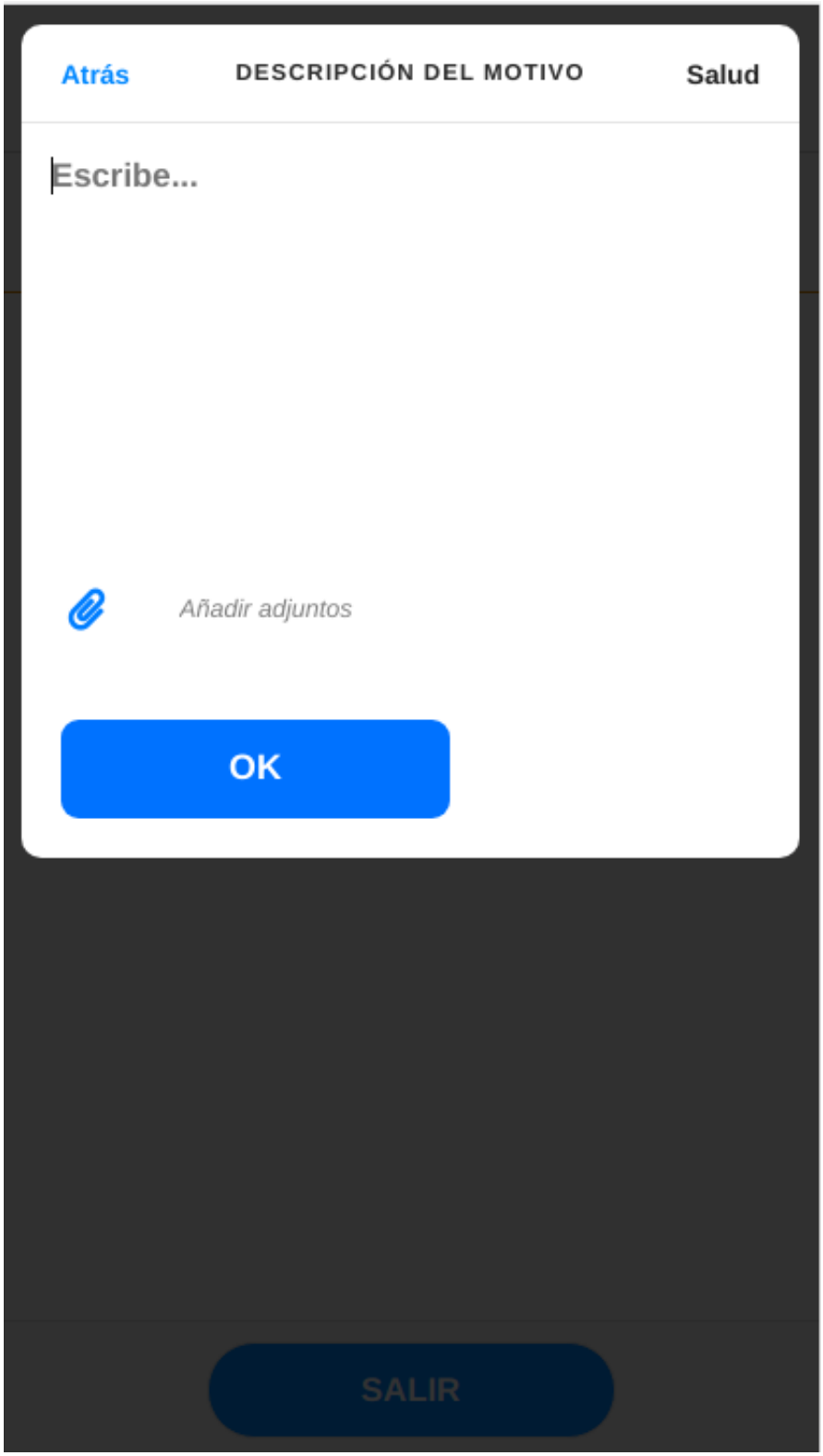
`@Input() placeholder: string = 'Escribe…'`

`@Input() icono_adicional: string = undefined`

`@Output() eventoTerminado = new EventEmitter()`

`@Output() eventoAdicional = new EventEmitter()`

`@Output() eventoCerrar = new EventEmitter()`

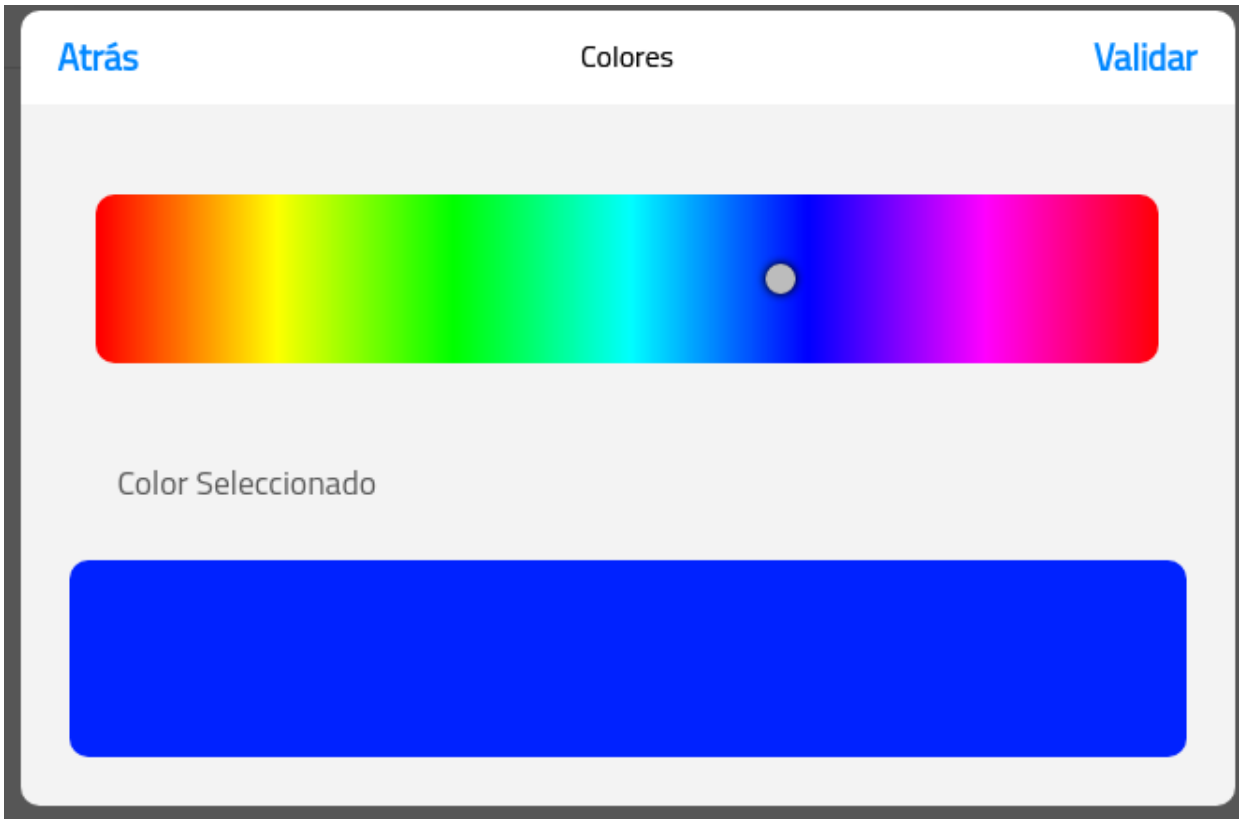


3.2.10 Color `app-color`

`@Input() color;`

`@Output() cerrar = new EventEmitter();`

`@Output() escribircolor = new EventEmitter();`



3.2.11 Resumen Enviado

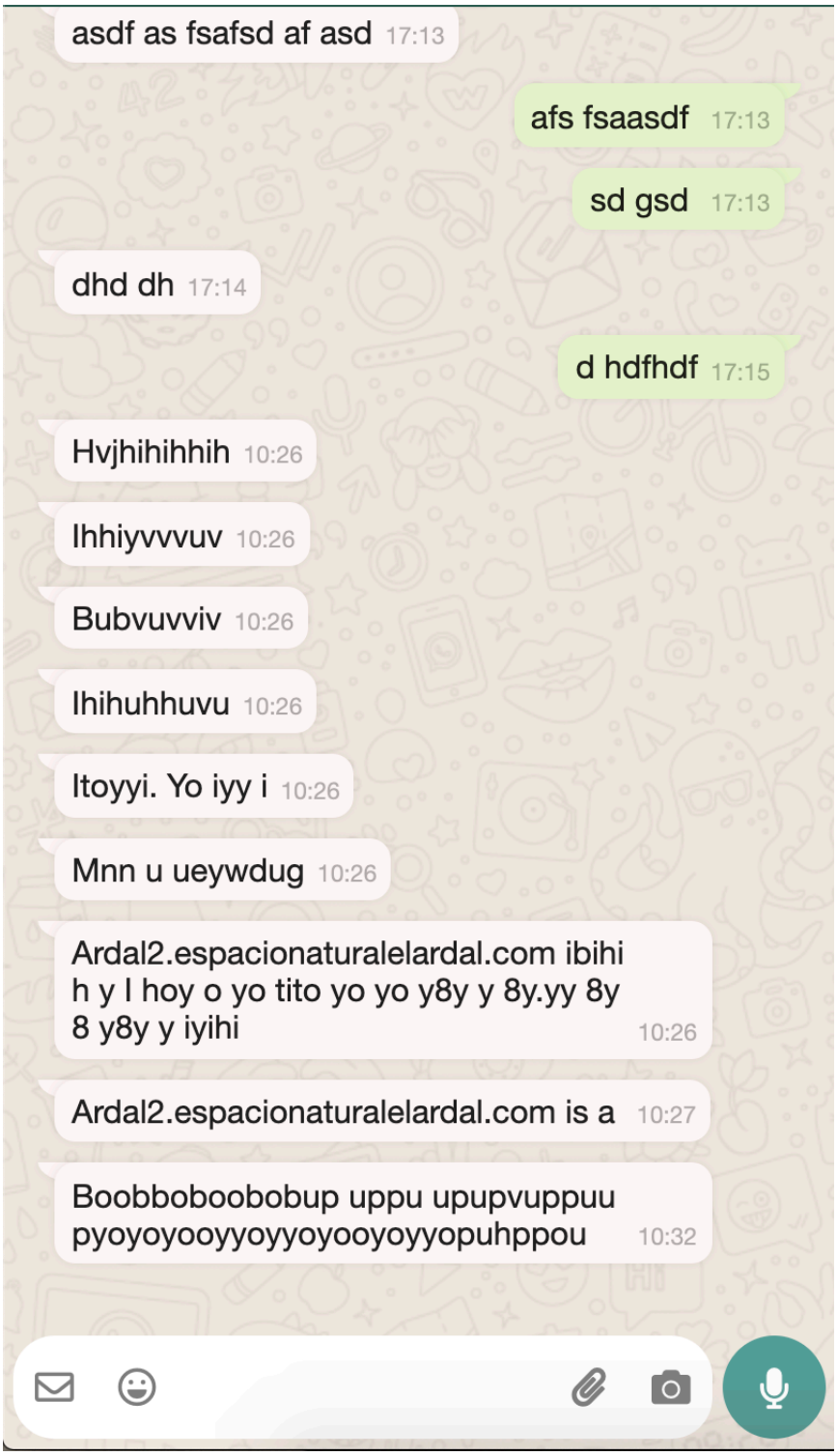
> Este es el diseño para los objetos tipo resumen enviado, será tipo emergente, en cada objeto su contenido sera diferente, pero el diseño exterior y colores seran los descritos. El padre que instancie este objeto deberá poner una capa desabilitadora oscura y deberá poder cerrarse con el swipeleft aparte del cerrar de arriba.

```
<div class="contenedor" (swipeleft)="cerrar.emit()">
    <div class="cerrar" (click)="cerrar.emit()">
      <i class="fa fa-times"></i>
    </div>
    <div class="cabecera">
    </div>
    <div class="cuerpo">

       <div class="contenido-boton">
          <div class="boton">
             <div   class="enviadocorrecto">
                <div  class="notificacion verde"><i class="fa fa-check">
</i>Petición enviada</div>
                <div  class="notificacion">Se ha enviado una notificación al
tutor</div>
             </div>
          </div>
       </div>
    </div>
</div>
```

```
:host{
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    z-index: 1;
    padding: 20px 10px;
    background: transparent;
    .contenedor{
          position: relative;
          width: 100%;
          height: 100%;
           display: block;
           border: 2px solid green;
           border-radius: 10px;
        .cerrar{
          width: 40px;
          height: 40px;
          position: absolute;
          top: -10px;
          right: -10px;
          display: flex;
          justify-content: center;
          align-items: center;
          background: #fff;
          border-radius: 10000px;
          font-size: 1.9rem;
          border: 1px solid #ccc;
          z-index: 2;
          -webkit-text-stroke: 1px white;
       }
       .cabecera{
          width: 100%;
          height: 75px;
          position: relative;
          display: block;
          background: #e0f9d7;
          border-bottom: 1px solid #ccc;
          z-index:1;
          border-top-left-radius: 10px;
          border-top-right-radius: 10px;

       }
       .cuerpo{
```

```
            width: 100%;
            height: calc(100% - 75px);
            position: relative;
            background: #f1ffec;
            border-bottom-left-radius: 10px;
            border-bottom-right-radius: 10px;
            .contenido-boton{
               height: 100px;
               width: 100%;
               position: absolute;
               bottom: 0;
               left: 0;
               display: -webkit-box;
               display: flex;
               -webkit-box-pack: center;
               justify-content: center;
               -webkit-box-align: center;
               align-items: center;
               background-color: #f1ffec;
               border-bottom-left-radius: 10px;
               border-bottom-right-radius: 10px;
               border-top: 1px solid #daead4;

               .boton{
                  width: 100%;
                  height: 100%;
                  display: flex;
                  justify-content: center;
                  align-items: center;
                  text-align: center;

                 .enviadocorrecto{
                     .notificacion{
                        font-size: 1.4rem;
                        i{
                           margin-right: 5%;
                        }
                        &.verde{
                           font-size: 2rem;
                           color: #0fbc0f;
                           margin-bottom: 10px;
                        }
                     }
                  }
               }
            }
         }

      }
```

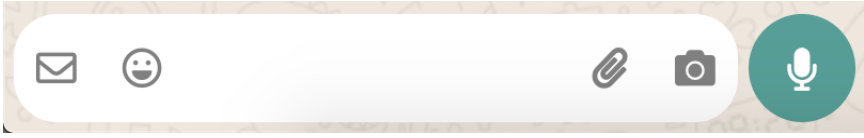
3.2.12 Whatsapp `app-whatsapp`



Este objeto esta compuesto de otros sub-objetos y actúa de padre entre ellos. Es importante esta actuación porque sin ella, los sub-objetos no funcionarían correctamente. Estos sub-objetos son: `app-entrada`, `app-mensajeria` y `app-mail`.

Primero vamos a ver los detalles de cada sub-objeto

`app-entrada`



Este objeto es el elemento de entrada donde el usuario introducirá los datos (texto, imagen, video, etc...). Consta de los siguientes atributos:

`@Output() otrosAdjuntos = new EventEmitter();` Se lanza cunado se pulsa el boton del `clip` `@Output() camara = new EventEmitter();` Se lanza cuando se pulsa el boton de la camara `@Output() cambioHeight = new EventEmitter();` Se lanza cuando cambia el `height` del propio objeto, esto ocurre cuando escribimos texto de varias lineas. El evento también lanza el nuevo `height` en `px` que ha obtenido. Esto es muy importante porque con este valor, el padre debe ajustar los demás elementos de la vista. Es muy importante tener eso en cuenta en caso de utilizar este objeto por separado. `@Output() envioTexto = new EventEmitter();` Se lanza cuando se pulsa el boton de enviar. `@Output() startGrabacion = new EventEmitter();` Se lanza cuando se inicia una grabación de audio, por lo tanto es el padre el que se encarga de realizar la propia grabación. `@Output() stopGrabacion = new EventEmitter();` Se lanza cuando se quiere terminar una grabación. `@Output() mostrarMail = new EventEmitter();` Se lanza cuando se pulsa el boton de mail.

> Como se puede apreciar, el objeto es puramente fachada, no realiza ningúna lógica de peso, es el padre el que se tiene que encargar de adjuntar archivos, de realizar las grabaciones de audio, etc...

`app-mensajeria`

Este objeto se encarga de mostrar los mensajer en un estilo parecido a `WhatsApp`. Consta de los siguientes atributos:

`@Input() mensajes: any = [];` El array de mensajes que debe mostrar. Este array debe contener objetos que a su vez tiene que tener la siguiente estructura:

```
mensaje: any = {
    /* emisor:
     * El id del usuario que envia el mensaje.
     * Es importante para saber de que lado poner los bocadillos
     */
    emisor
    /* tipo:
     * 0 -> Texto
     * 1 -> Adjunto (imagen, video, etc...)
```

```
     * 2 -> Mail
     */
    tipo
    /* tipoadjunto:
     * Solo si tipo = 1
     * 0 -> Imagen
     * 1 -> Video (no implementado)
     * 2 -> Audio
     * 3 -> Documento (no implementado)
     */
    tipoadjunto
    texto
    hora
    /* cargando:
     * Utilizamos esta variable en mensajes con adjuntos
     * para indicar una animacion mientras se carga el adjunto
     */
    cargando
    adjunto: any = {
        local   // TRUE si la imagen no viene del servidor
        adjunto // URL del archivo
    }
};
```

`@Input() bajarScroll: boolean = false;` Fuerza al listado de mensajes desplazarse hasta la posición mas baja. `@Input() heightEntrada: string;` Este objeto necesita saber el alto del objeto `app-entrada` para poder ajustar bien los mensajes cuando dicho objeto crece. Tener en cuenta que en cierta circunstancia el valor de este atributo no tiene que ser el del objeto `app-entrada`.

> Como se puede apreciar, el objeto es puramente fachada, no realiza ningúna lógica de peso, es el padre el que se tiene que encargar de gestionar el array de mensajería.

`app-mail`

Es simplemente un objeto que recoge los datos para realizar un envio de mail, es autosuficiente y no necesita ningún cuidado especial. Consta de los siguientes atributos:

`@Output() eventoCerrar = new EventEmitter();` `@Output() eventoTerminar = new EventEmitter();` Emite un objeto con los datos del mail creado, este objeto tiene la siguiente estructura `{asunto: ..., comentario: ..., adjuntos: []}`

Vistos estos sub-objetos por separado, ahora vamos a ver la función que realiza `app-whatsapp`. Como he mencionado anteriormente, los sub-objetos son básicamente vistas, y no realizan mucha lógica, por lo tanto necesitan de un padre que realice la función de hilo conductor entre ellos. Se encarga practicamente de lo más importante, que es leer los mensajes, enviar los

mensajes, enviar adjuntos, realizar las grabaciones de audio, etc...

Este objeto está pensado para funcionar siempre imprescindiblemente con los sub-objetos `app-entrada` y `app-mensajeria`. Estos dos objetos son necesarios obligatoriamente para que funcione correctamente. El sub-objeto `app-mail` es básicamente un añadido y no es imprescindible. Esto que estoy remarcando es importante porque en ciertas circunstancias es posible que queramos implementar **solo** el objeto `app-entrada` o **solo** el objeto `app-mensajeria`, y para que estos objetos funcionen bien por separado, tenemos que tener en cuenta todos sus atributos y realizar la lógica que realiza en este caso `app-whatsapp` allí donde los vayamos a instanciar. Nos podemos guiar con el código de `app-whatsapp` para realizar esta tarea.

4. Protocolo multimedia (assets)

Todos los archivos multimedia que un proyecto utilice debe situarse bajo la carpeta `/assets/`. La carpeta debe estar organizada según el siguiente esquema:

```
project
└─── assets
        └─── img        // Archivos de imagen
        └─── video      // Archivos de video
        └─── audio      // Archivos de audio
        └─── fonts      // Fuentes
        └─── js         // Scripts javascript
        └─── css        // Ficheros de estilo
        .
        .
```

4.1 Imágenes

Todas las imágenes deben guardarse bajo el directorio `/assets/img/`. Si las imágenes forman parte de un módulo, deberán guardarse bajo el directorio `/assets/img/NOMBRE_MODULO/`.

Las imágenes necesitan un tratamiento para conseguir un rendimiento adecuado. Este tratamiento consta de **redimensionado** y de **compresión**.

4.1.1 Redimensionado

Medir la imagen en el proyecto en píxeles (ancho x alto) y redimensionar la imagen a esa resolución como máximo.

¿Como se mide la imagen?

Debemos ejecutar el proyecto con la imagen, insepccionar el elemento (`F12`) en una vista donde mayor se vea la imagen, y observar el tamaño en píxeles que nos da el inspeccionado (`F12`).

4.1.2 Compresión

Después de redimensionar la imagen, lo que debemos hacer es comprimirla para obtener el mínimo tamaño posible. La herramienta mas sencilla para hacer eso es la siguiente web `https://imagecompressor.com/`. Lo que debemos hacer es subir las imagenes a la web y reducir su calidad como mínimo al **50%**. En ciertas circunstancias sería posible reducirla más todavia, pero en todo caso debemos asegurarnos que el resultado sea aceptable.

Actualizaciones

- 16/06/2020: Documentadas capas deshabilitadoras `seccón 3.1.4`.
- 18/06/2020: Añadida cambio en la clase `boton-cerrar` en `sección 3.2.1` sobre el tamaño del boton en móviles y tablets. Añadido nuevo efecto de Pop con rebote en `sección 2.4.1`.
- 25/06/2020: Actualizado objeto de Pizarra `sección 3.2.9`.
- 26/06/2020: Actualizado objeto FechahoraPicker `sección 3.2.3`. Actualización de la `sección 2.2`.
- 27/06/2020: Arreglado errores en objeto `desplegable` `sección 3.2.8`.
- 02/07/2020: Actualizado objeto FechahoraPicker `sección 3.2.3`.
- 20/07/2020: Actualizado objeto Color `seccion 3.2.10` elemento swich `seccion 3.1.5`.
- 21/07/2020: Añadida documentación sobre boton atrás de android `seccion 2.4.2`.
- 27/07/2020: Añadida documentación sobre boton validar `seccion 3.2.6`, boton enviar y atras `sección 3.2.7`. Objeto resumen enviado `sección 3.2.11`
- 29/07/2020: Añadida documentacion sobre statusbar `seccion 2.4.3` , botonera tipo 1 `sección 3.2.9`

  30/07/2020: Añadida documentacion sobre elementos pildora `seccion 3.2.10` y filtrar por texto `seccion 2.4.4`
- 11/09/2020: Añadida documentación sobre objeto de whatsapp y sus sub-objetos (entrada, mensajería y mail) `seccion 3.2.12`
- 20/09/2020: Añadida documentación sobre el protocolo de multimedia `seccion 4`. Documentadas estilo de fechas `sección 1.1.4`. Actualizado objeto resumen enviado, también se puede cerrar con swipeleft `sección 3.2.11`
- 05/11/2020: Complementada documentación de efectos en `sección 2.4.1`
- 09/11/2020: Añadido elementos de `caja` y `listado`, `sección 3.1.11` y `sección 3.1.12`